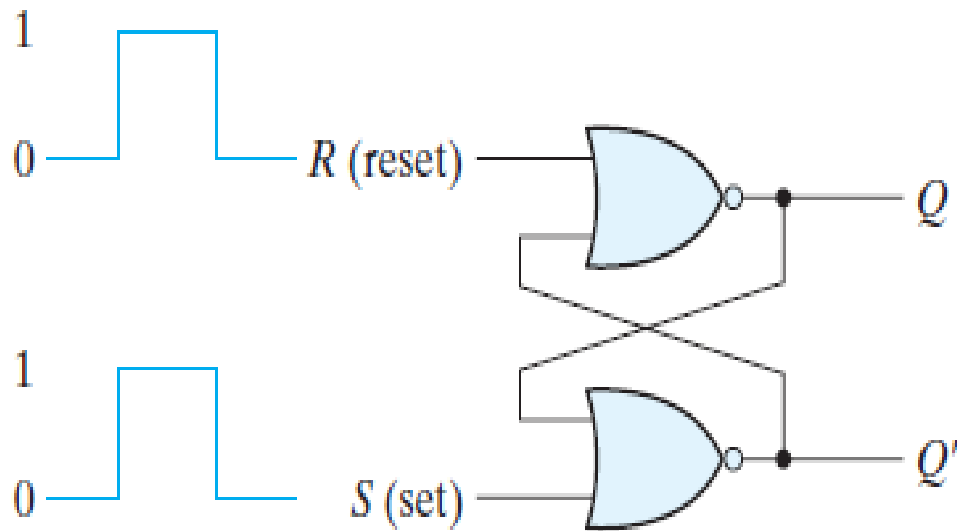


Sequential Circuits

Unit – 5

Lecture by
Vikram Kulkarni

SR latch with NOR gates

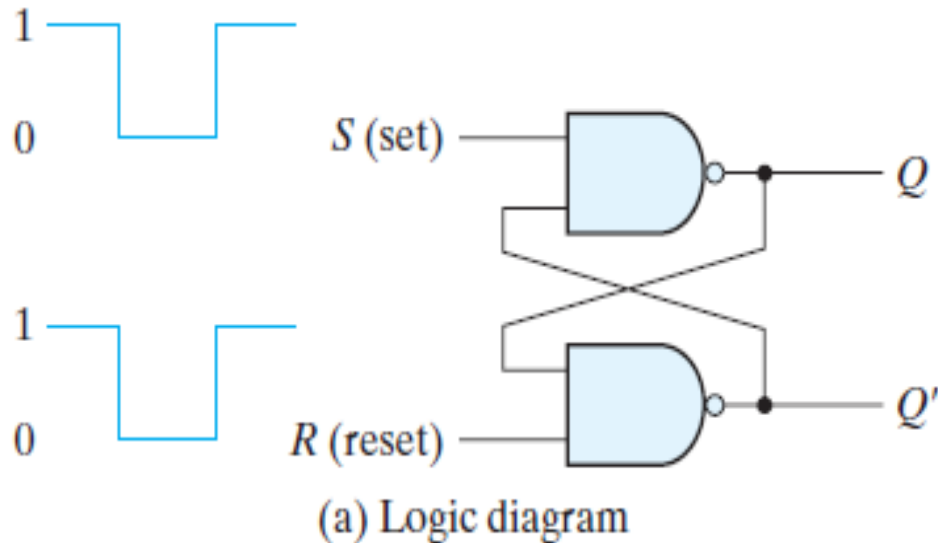


(a) Logic diagram

S	R	Q	Q'
1	0	1	0
0	0	1	0 (after $S = 1, R = 0$)
0	1	0	1
0	0	0	1 (after $S = 0, R = 1$)
1	1	0	0 (forbidden)

(b) Function table

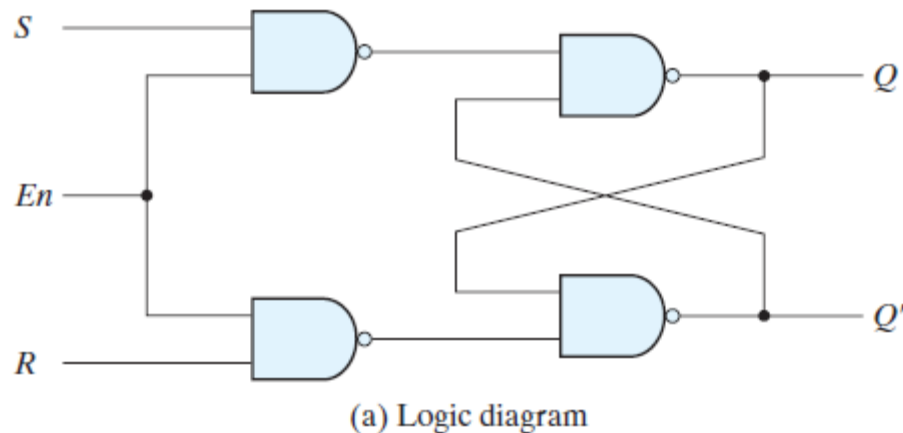
SR latch with NAND gates



S	R	Q	Q'
1	0	0	1
1	1	0	1 (after $S = 1, R = 0$)
0	1	1	0
1	1	1	0 (after $S = 0, R = 1$)
0	0	1	1 (forbidden)

(b) Function table

SR FlipFlop



Functional Table

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

Characteristic table and excitation table of SR Flip-flop

Characteristic Table

Q_n	S	R	Q_{n+1}
0	0	0	0 i.e., Q_n
0	0	1	0
0	1	0	1
0	1	1	×
1	0	0	1 i.e., Q_n
1	0	1	0
1	1	0	1
1	1	1	×

Excitation Table

Q_n	Q_{n+1}	S	R
0	0	0	×
0	1	1	0
1	0	0	1
1	1	×	0

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

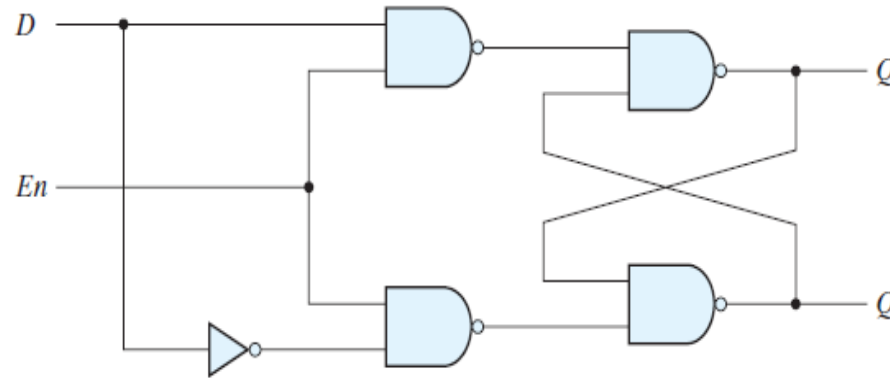
Using K-Map we obtain

$$Q_{n+1} = S + \bar{R} Q_n$$

D Flip flop

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table



(a) Logic diagram

Functional Table

En	D	Next state of Q
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

(b) Function table

Characteristic Table

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

Excitation Table

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

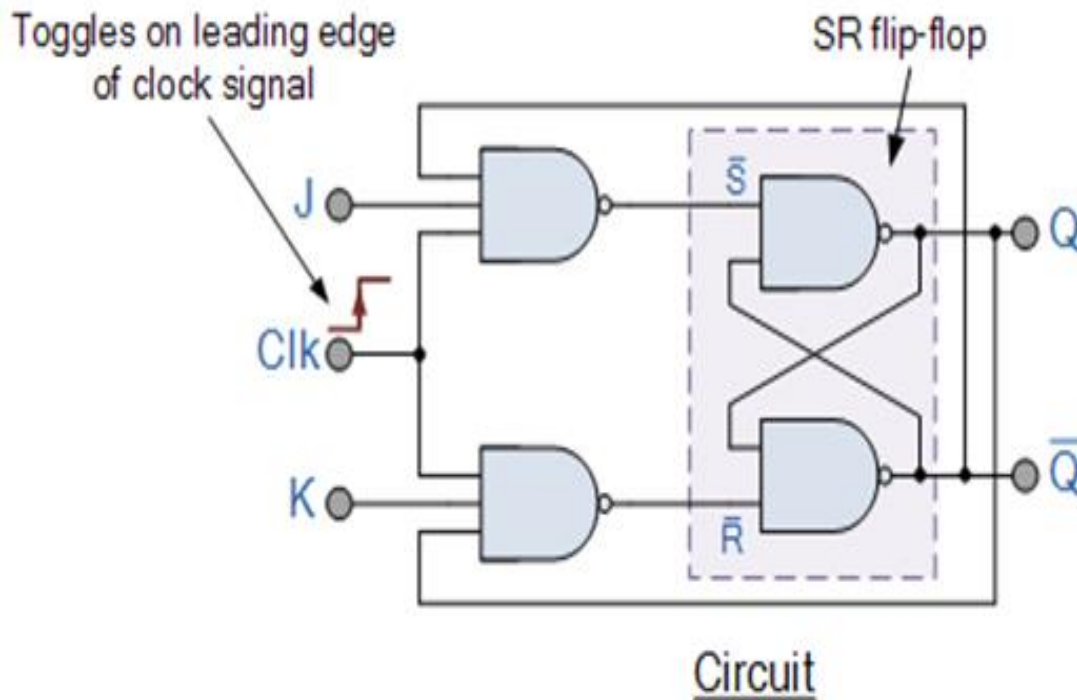
Using K-Map
we obtain

$$\Rightarrow Q_{n+1} = D$$

JK Flip Flop

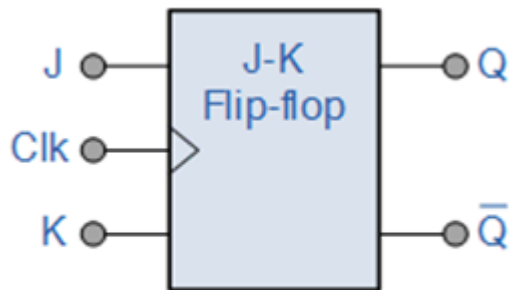
- The basic S-R NAND flip-flop circuit has many advantages and uses in sequential logic circuits but it suffers from two basic switching problems.
 1. the Set = 1 and Reset = 1 condition ($S = R = 1$) must always be avoided
 2. if Set or Reset change state while the enable (EN) input is high the correct latching action may not occur
- Then to overcome these two fundamental design problems with the SR flip-flop design, the **JK flip Flop** was developed.

The Basic JK Flip-flop



Functional Table

Clk	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Toggle $\overline{Q_n}$



SR-FF			
En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

SR Latch - NAND

S	R	Q	Q'
1	0	0	1
1	1	0	1 (after $S = 1, R = 0$)
0	1	1	0
1	1	1	0 (after $S = 0, R = 1$)
0	0	1	1 (forbidden)

Characteristic table and excitation table of JK Flip-flop

SR-FF

En	S	R	Next state of Q
0	X	X	No change
1	0	0	No change
1	0	1	$Q = 0$; reset state
1	1	0	$Q = 1$; set state
1	1	1	Indeterminate

(b) Function table

SR Latch - NAND

S	R	Q	Q'
1	0	0	1
1	1	0	1 (after $S = 1, R = 0$)
0	1	1	0
1	1	1	0 (after $S = 0, R = 1$)
0	0	1	1 (forbidden)

Using K-Map we obtain

$$Q_{n+1} = \overline{Q}_n J + Q_n \overline{K}$$



JK FF -Characteristic Table

Q_n	J	K	Q_{n+1}
0	0	0	0 i.e., Q_n
0	0	1	0
0	1	0	1
0	1	1	1 i.e., $\overline{Q}_n$
1	0	0	1 i.e., Q_n
1	0	1	0
1	1	0	1
1	1	1	0 i.e., $\overline{Q}_n$

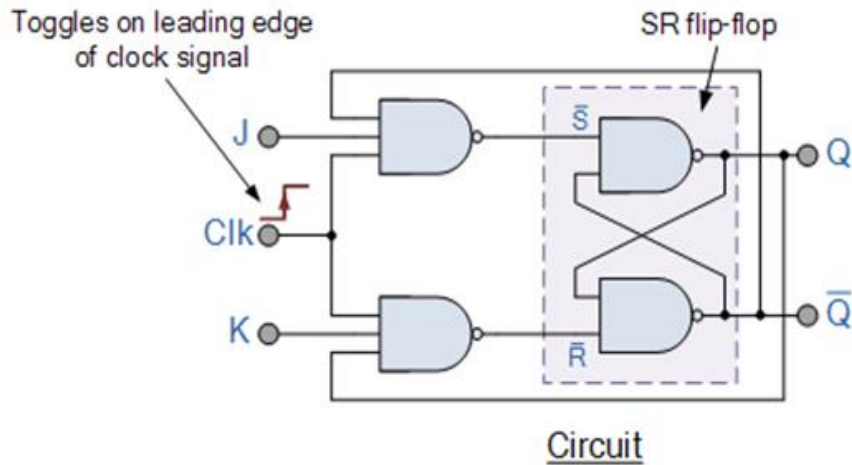
JK- FF _ Functional Table

Clk	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	$\overline{Q}_n$ Toggle

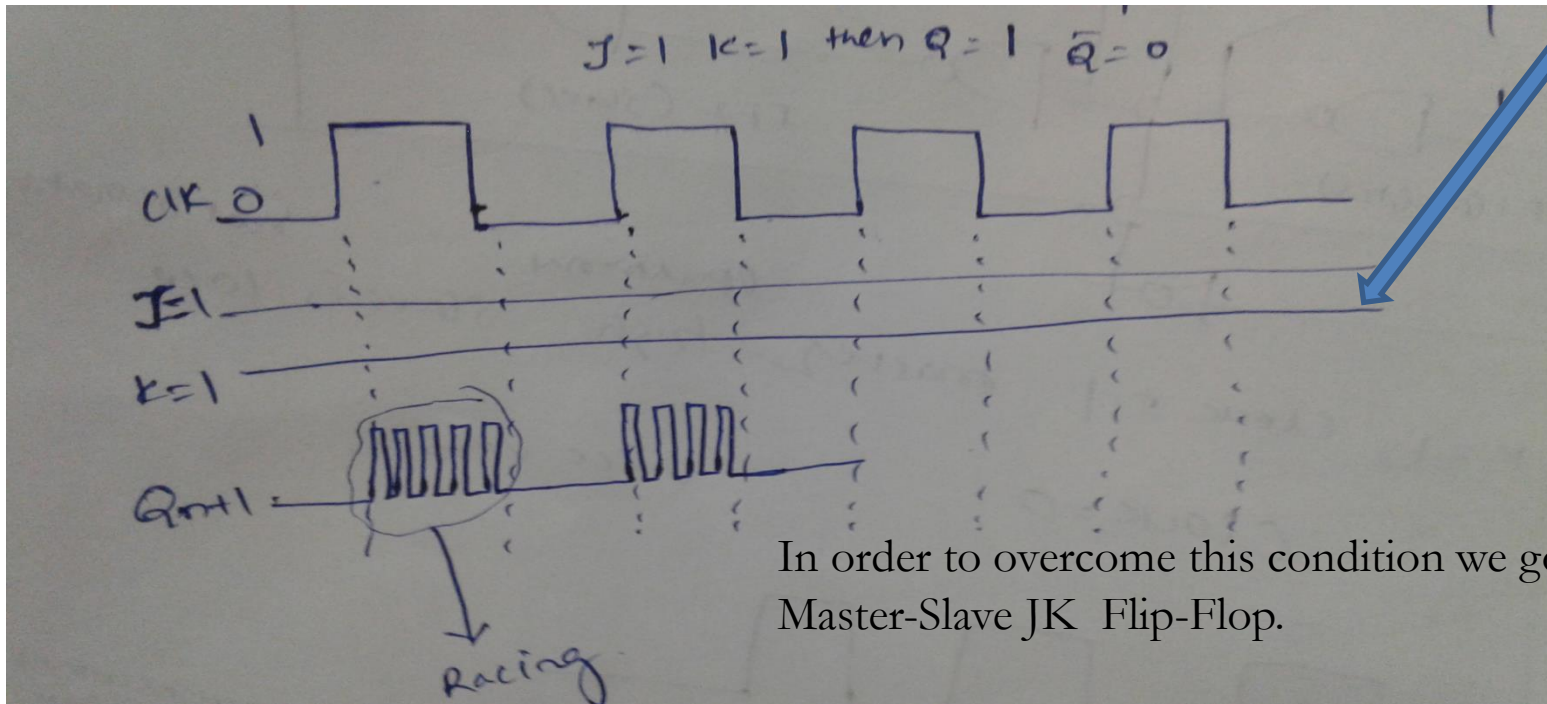
JK FF -Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

JK Flip-flop – Race Around Condition



Clk	J	K	Q_{n+1}	$\overline{Q}_{n+1}$
0	X	X	Q_n	$\overline{Q}_n$
1	0	0	Q_n	$\overline{Q}_n$
1	0	1	0	1
1	1	0	1	0
1	1	1		

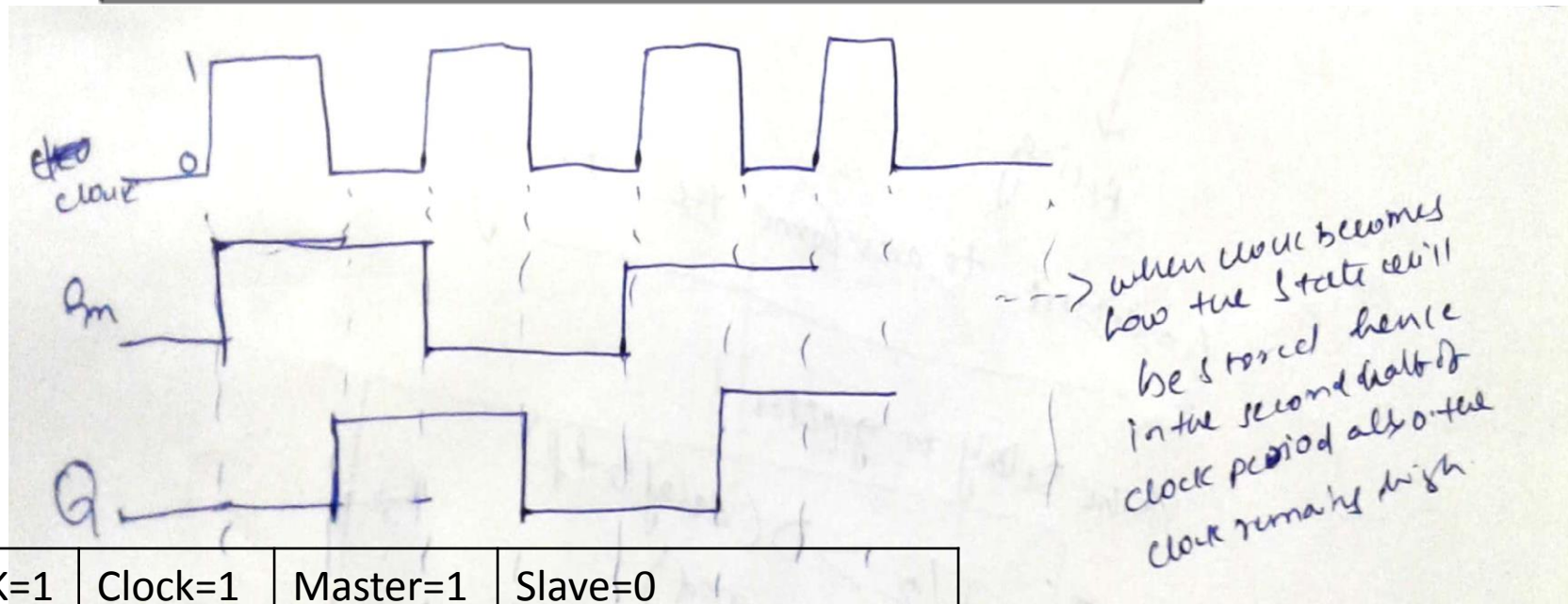
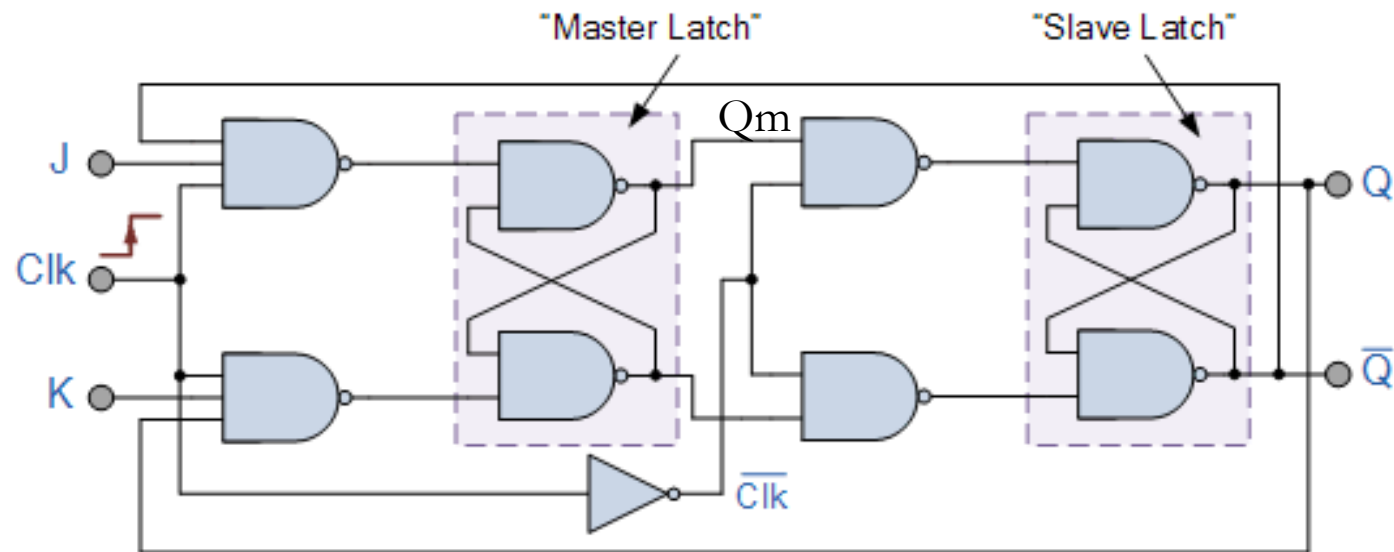


In order to overcome this condition we go for Master-Slave JK Flip-Flop.

The Master-Slave JK Flip-flop

- The **Master-Slave Flip-Flop** is basically two gated SR flip-flops connected together in a series configuration with the slave having an inverted clock pulse.
- The outputs from Q and Q' from the “Slave” flip-flop are fed back to the inputs of the “Master” with the outputs of the “Master” flip flop being connected to the two inputs of the “Slave” flip flop.
- This feedback configuration from the slave’s output to the master’s input gives the characteristic toggle of the JK flip flop as shown below.

The Master-Slave JK Flip Flop

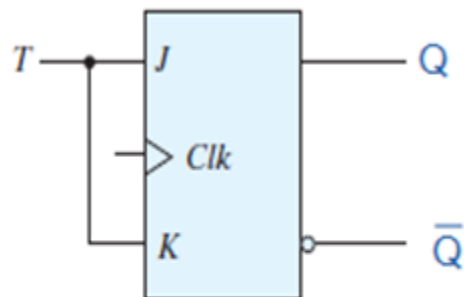


J=1	K=1	Clock=1	Master=1	Slave=0
J=1	K=1	Clock=0	Master=0	Slave=1

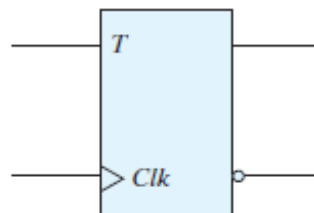
The Master-Slave JK Flip Flop

- The input signals J and K are connected to the gated “master” SR flip flop which “locks” the input condition while the clock (Clk) input is “HIGH” at logic level “1”. As the clock input of the “slave” flip flop is the inverse (complement) of the “master” clock input, the “slave” SR flip flop does not toggle. The outputs from the “master” flip flop are only “seen” by the gated “slave” flip flop when the clock input goes “LOW” to logic level “0”.
- When the clock is “LOW”, the outputs from the “master” flip flop are latched and any additional changes to its inputs are ignored. The gated “slave” flip flop now responds to the state of its inputs passed over by the “master” section.
- Then on the “Low-to-High” transition of the clock pulse the inputs of the “master” flip flop are fed through to the gated inputs of the “slave” flip flop and on the “High-to-Low” transition the same inputs are reflected on the output of the “slave” making this type of flip flop edge or pulse-triggered.

T flip-flop



(a) From JK flip-flop



(c) Graphic symbol

Functional Table

Clock	T	Q_{n+1}
0	x	Q_n (Memory)
1	0	Q_n (Memory)
1	1	$\overline{Q_n}$ (Toggling)

Characteristic Table

Q_n	T	Q_{n+1}
0	0	0 Q_n
0	1	1
1	0	1
1	1	0



$$Q_{n+1} = Q_n \oplus T$$

$$D = T \oplus Q = TQ' + T'Q$$

Excitation Table

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

FLIP-FLOP CONVERSIONS

<i>Desired transition</i>		<i>Triggering signal needed</i>					
Q_n	Q_{n+1}	S	R	J	K	D	T
0	0	0	x	0	x	0	0
0	1	1	0	1	x	1	1
1	0	0	1	x	1	0	1
1	1	x	0	x	0	1	0

Flip-Flop Conversion

Steps for conversion:

1. Identify available and required FF
2. Make Characteristic table for required FF
3. Make Excitation table for available FF
4. Write the Boolean Expression for available FF
5. Draw the Circuit.

JK FF - to - D FF conversion

Characteristic table of D FF

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

JK FF -Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	×
0	1	1	×
1	0	×	1
1	1	×	0

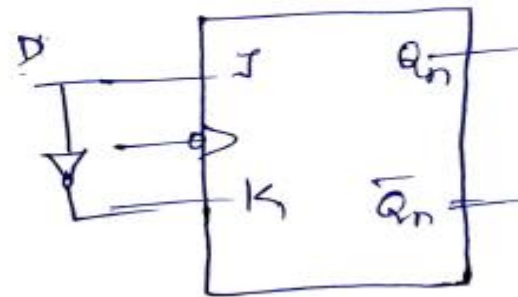
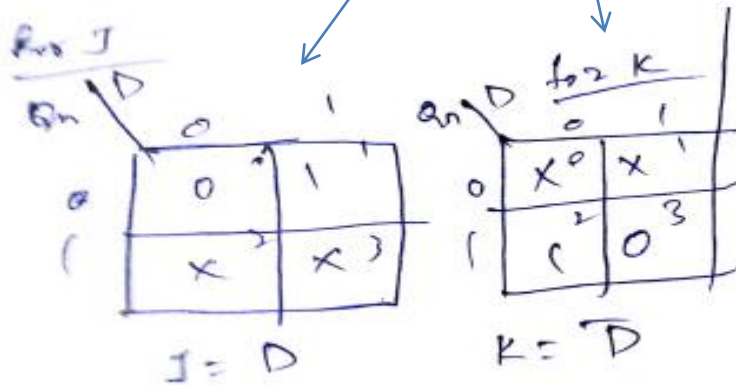
JK FF - to - D FF conversion

Characteristic table of D FF

Q_n	D	Q_{n+1}	J	K
0	0	0	0	x
0	1	1	1	x
1	0	0	x	1
1	1	1	x	0

JK FF -Excitation Table

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

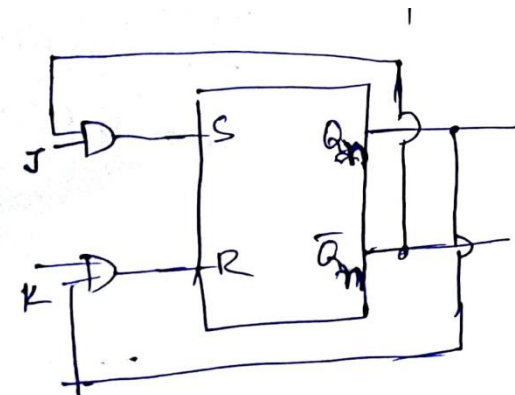
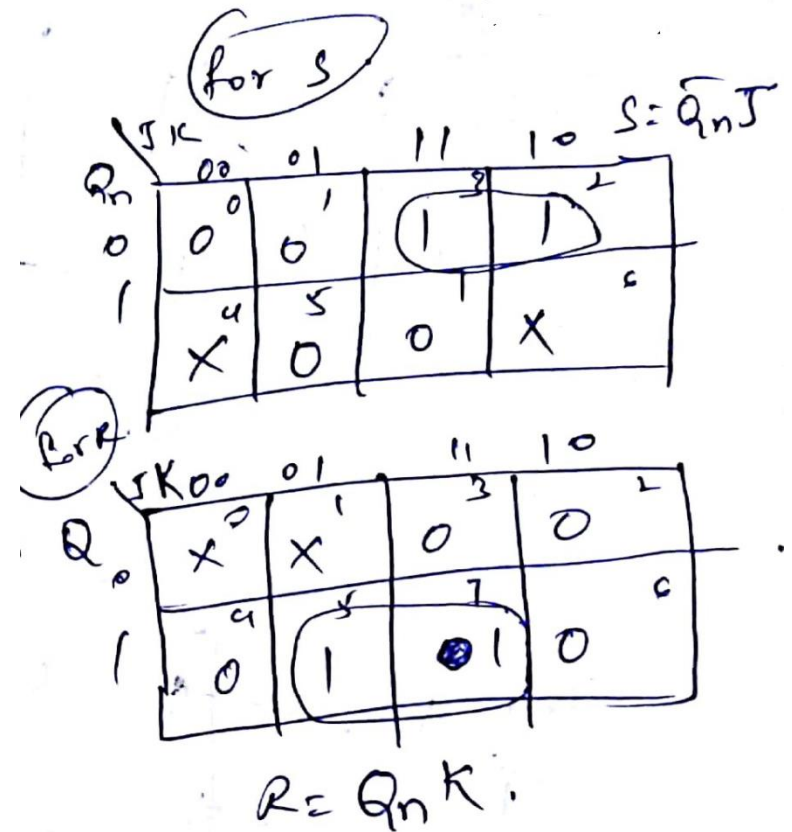


Convert a SR-FF to a JK-FF.

Q_n	J	K	Q_{n+1}	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

Excitation Table

Q_n	Q_{n+1}	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

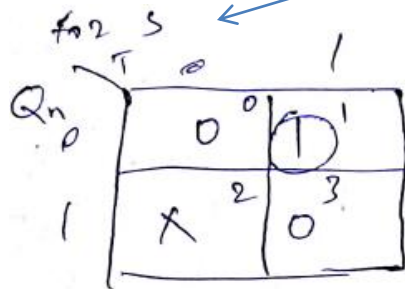


SR-FF to T-FF Conversion

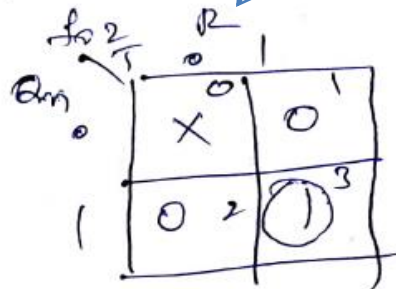
Q_n	T	Q_{n+1}	S	R
0	0	0	0	x
0	1	1	1	0
1	0	1	x	0
1	1	0	0	1

Excitation Table

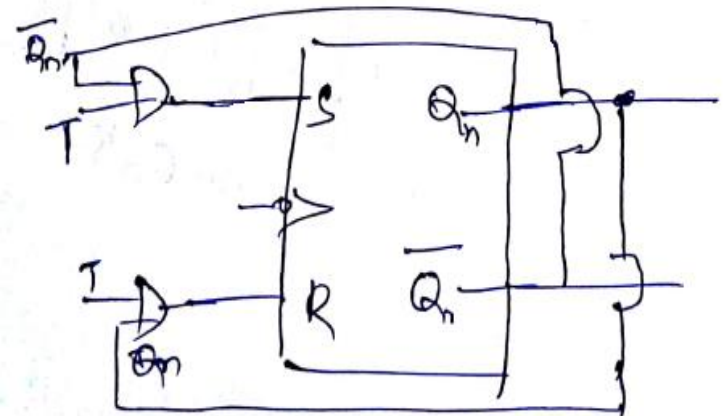
Q_n	Q_{n+1}	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0



$$S = \overline{Q_n} T$$



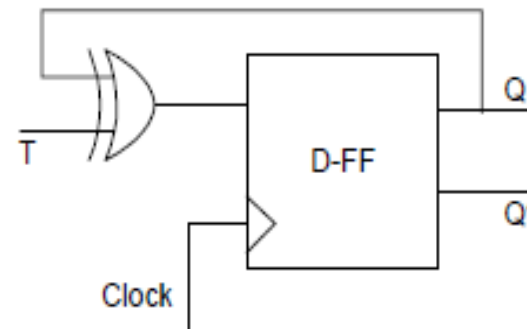
$$R = Q_n T$$



Convert a D-FF to a T-FF:

- We need to design the circuit to generate the triggering signal D as a function of T and Q : $D = f(T, Q)$

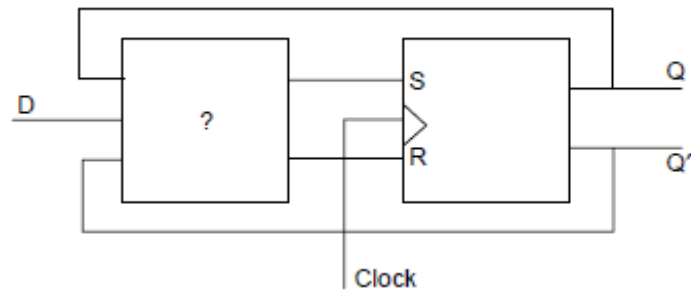
Q_n	Q_{n+1}	T	D
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	1



Treating D as a function of T and current FF state Q Q_n we have

$$D = T'Q + TQ = T \oplus Q$$

Convert a SR-FF to a D-FF:



Q_n	Q_{n+1}	D	S	R
0	0	0	0	X
0	1	1	1	0
1	0	0	0	1
1	1	1	X	0

The desired signal S and R can be obtained as functions of T and current FF state Q from the Karnaugh maps:

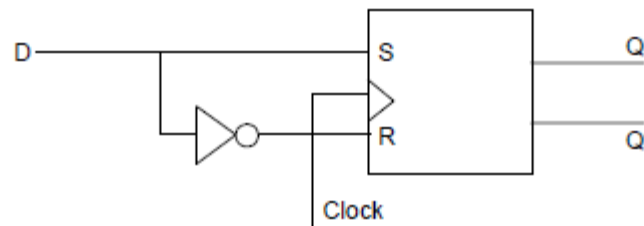
D \ Q	0	1
	0	0
0	0	0
1	1	X

$S = D$

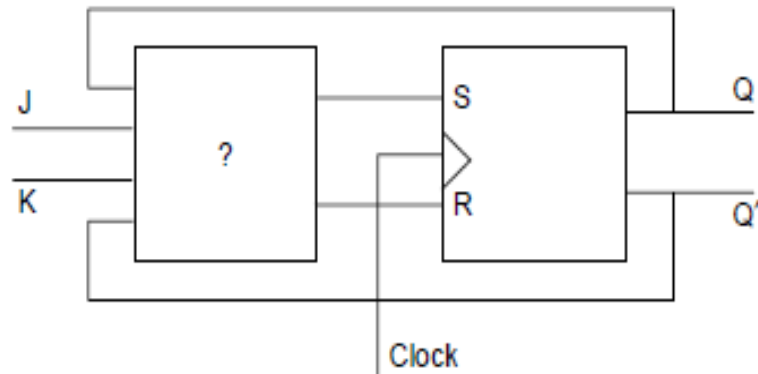
D \ Q	0	1
	0	1
0	X	1
1	0	0

$R = D'$

$$S = D, R = D'$$



Convert a SR-FF to a JK-FF.



Q_n	Q_{n+1}	J	K	S	R
0	0	0	x	0	x
0	1	1	x	1	0
1	0	x	1	0	1
1	1	x	0	x	0

The desired signals S and R as function J , K and current FF state Q can be obtained from the Karnaugh maps:

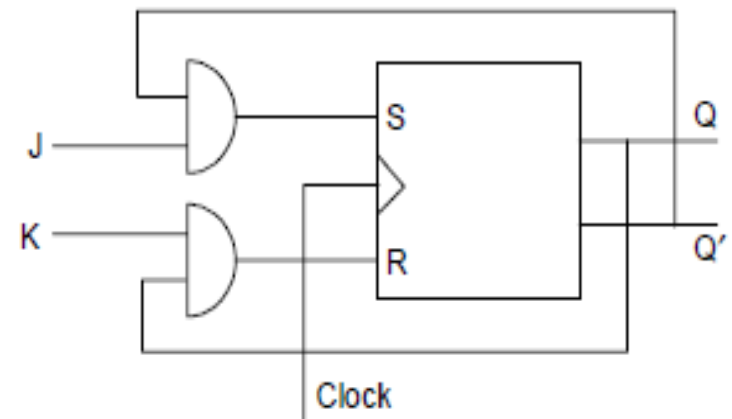
K \ QJ	00	01	11	10
0	0	1	x	x
1	1	1	0	0

$$S = Q'J$$

K \ QJ	00	01	11	10
0	x	0	0	0
0	x	0	1	1

$$R = QK$$

$$S = Q'J, \quad R = QK$$



SR Flip Flop to T Flip Flop

Input	Present state	Next state	Flip-Flop Input	
T	Q_n	Q_{n+1}	S	R
0	0	0	0	×
0	1	1	×	0
1	0	1	1	0
1	1	0	0	1

For s

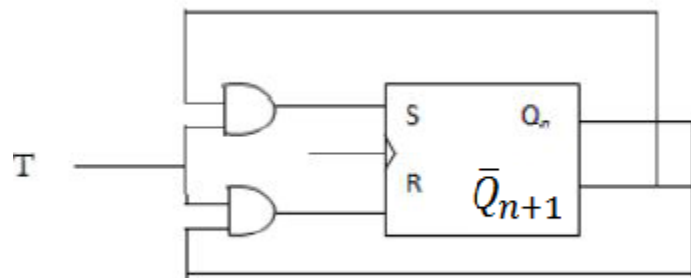
$T \backslash Q_n$	0	1
0	0	×
1	1	0

$s = T\bar{Q}_n$

for R

$T \backslash Q_n$	0	1
0	×	0
1	0	1

$R = TQ_n$

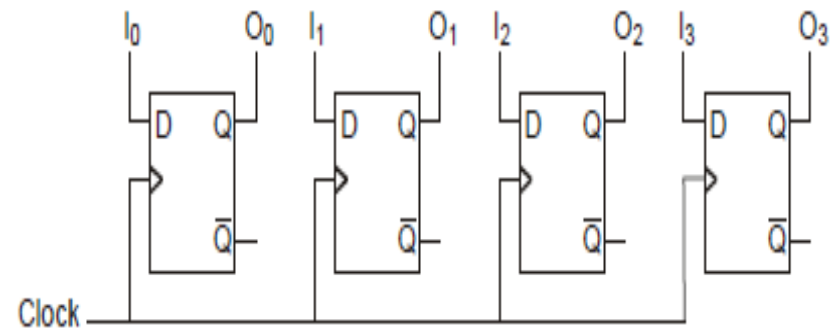


Registers and Counters

Lecture By: Vikram Kulkarni

Registers

- Registers are the group of flip-flops (single bit storage element).
- The simplest type of register is a data register, which is used for the temporary storage of data.
- In its simplest form, it consists of a set of N D flip-flops, all sharing a common clock.
- All of the digits in the N bit data word are connected to the data register by an N line “data bus”.



Shift Register

- A common form of register used in many types of logic circuits is a shift register.
- Registers like counters, are sequential circuits and as they employ flip-flops they possess memory; but memory is not the only requirement of a shift register.
- The function of storage of binary data can be very well performed by a simple register. Shift registers are required to do much more than that.
- They are required to store binary data momentarily until it is utilized for instance, by a computer, microprocessor, etc.
- Sometimes data is required to be presented to a device in a manner which may be different from the way in which it is fed to a shift register.
- For instance, shift register can present data to a device in a serial or parallel form, irrespective of the manner in which it is fed to a shift register.
- Data can also be manipulated within the shift register, so that it is presented to a device in the required form.
- These devices can also shift left or right and it is this capability which gives them the name of shift register.

Data conversion with a shift register

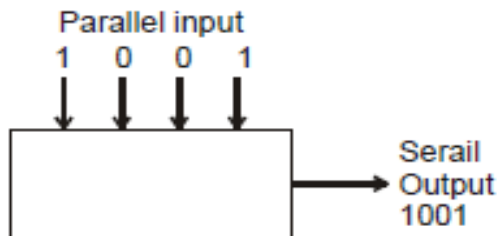


(a) Serial input/Serial output

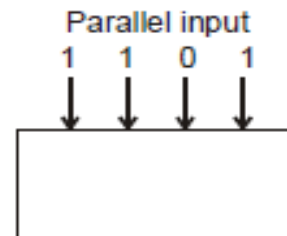


Parallel Output

(b) Serial input/Parallel output



(c) Parallel input/Serial output



Parallel Output

(b) Parallel input/Parallel output

Shift Registers with D FF

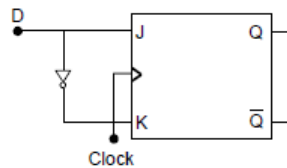


Fig. JK Flip-flop converted into D-Flip-Flop

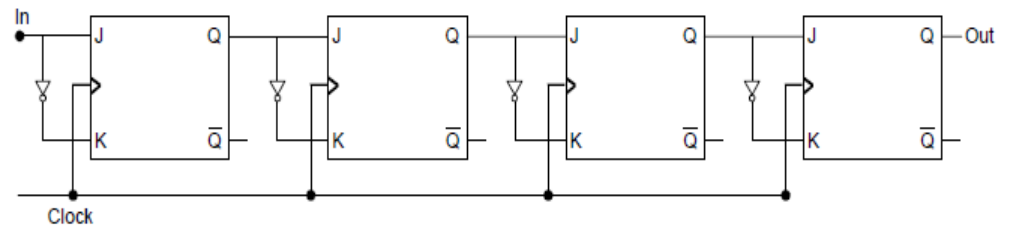


Fig. (a) 4-bit serial in serial out shift register using JK Flip-Flop

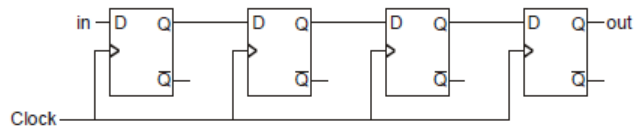


Fig. 4-bit serial-in serial-out shift register

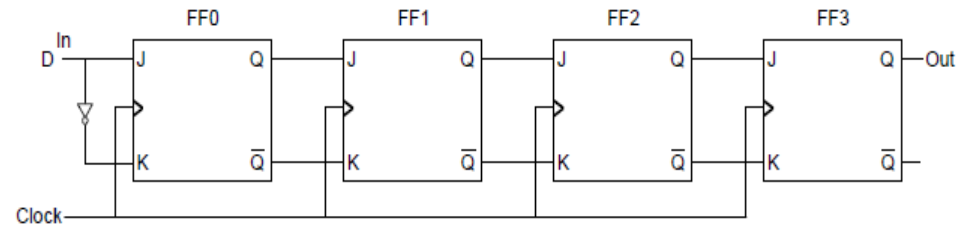


Fig. (b) 4-bit serial in-serial out shift register using JK Flip-flop

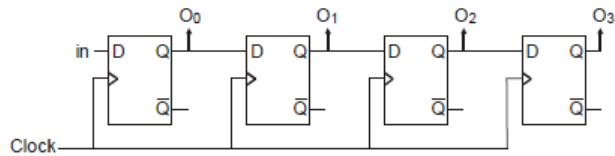
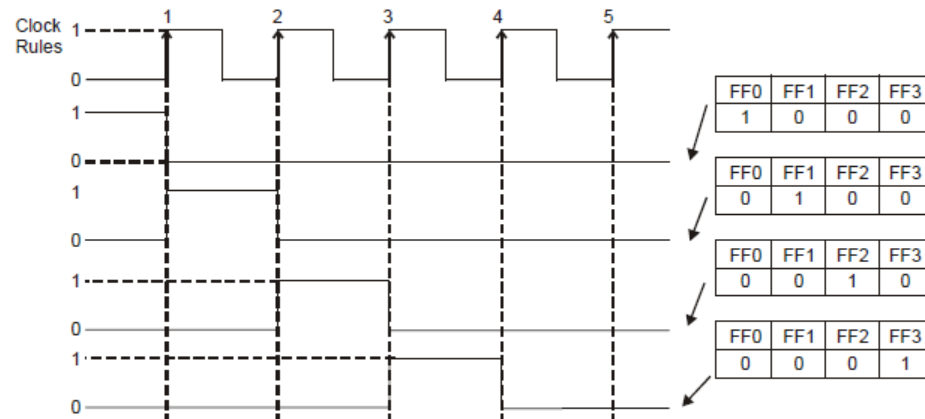
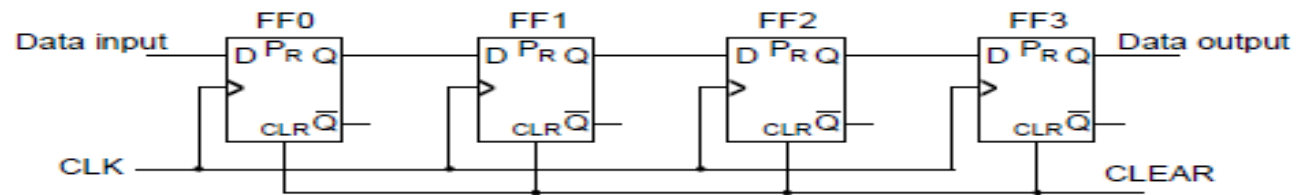


Fig. 4-bit serial-in parallel-out shift register

A 4-bit Shift Register operation

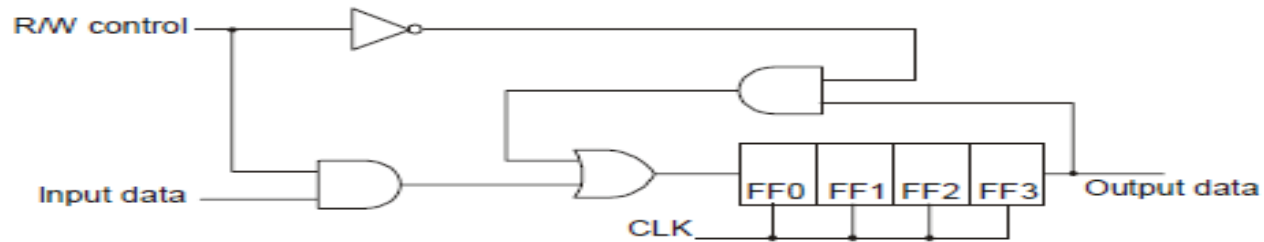


Serial In–Serial Out Shift Registers

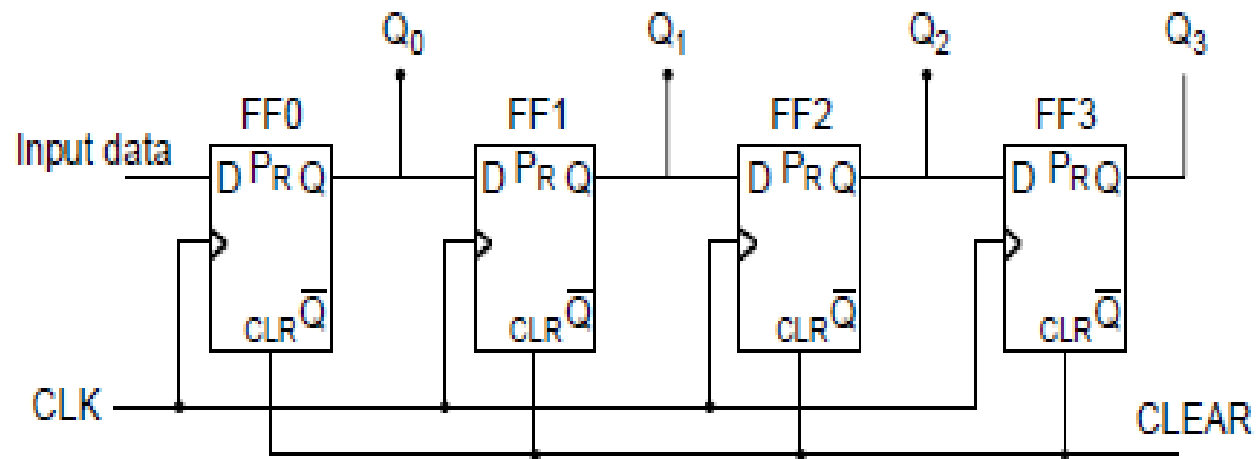


	FF0	FF1	FF2	FF3
CLEAR	0	0	0	0

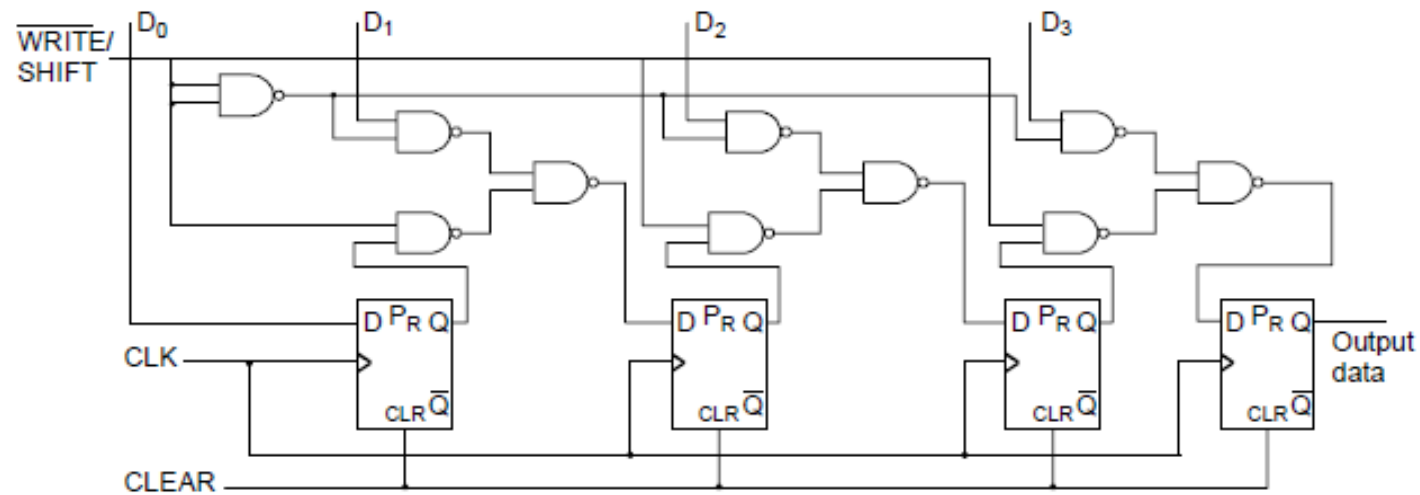
0000	FF0	FF1	FF2	FF3	0000
	1	0	0	1	



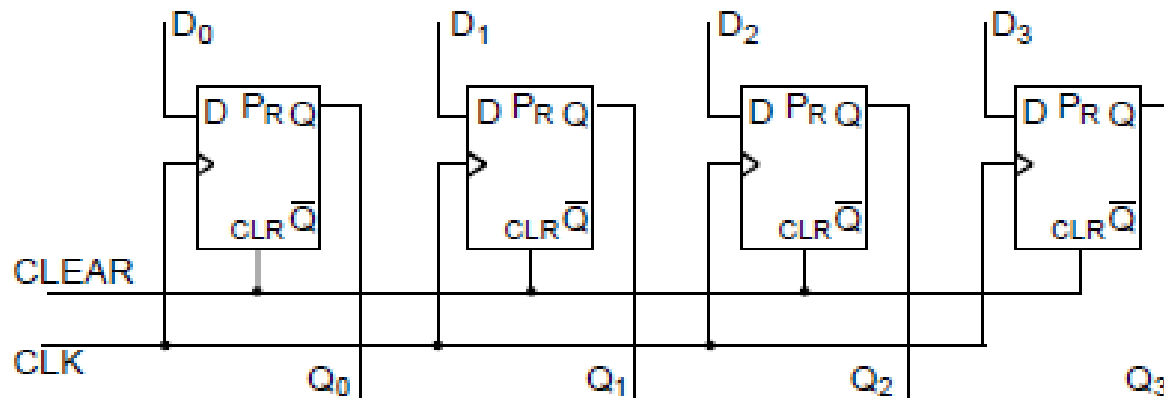
Serial In-Parallel Out Shift Registers



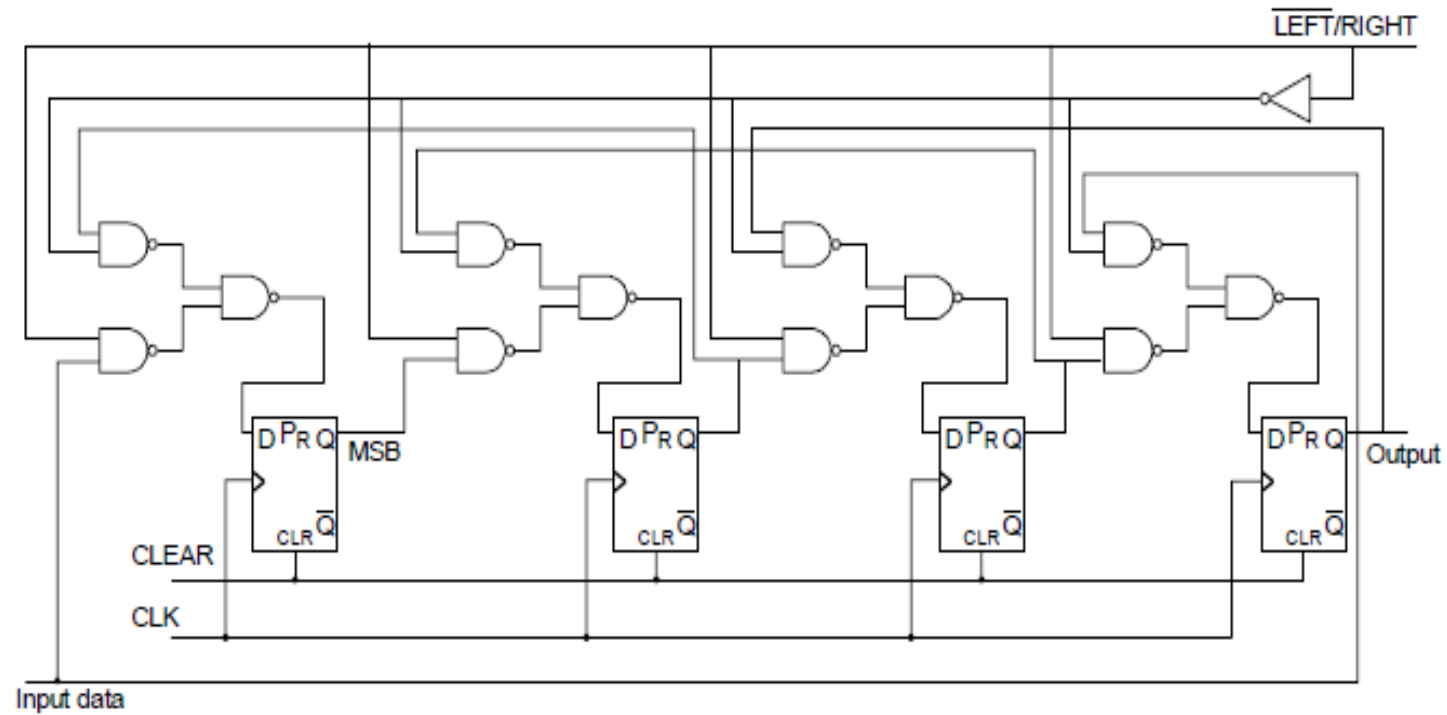
Parallel In-Serial Out Shift Registers



Parallel In-Parallel Out Shift Registers



Bidirectional Shift Registers (Universal Shift Register)



APPLICATIONS OF SHIFT REGISTERS

- To Produce Time Delay
- To Simplify Combinational Logic
- To Convert Serial Data to Parallel Data

COUNTERS

Binary Ripple Up-Counter

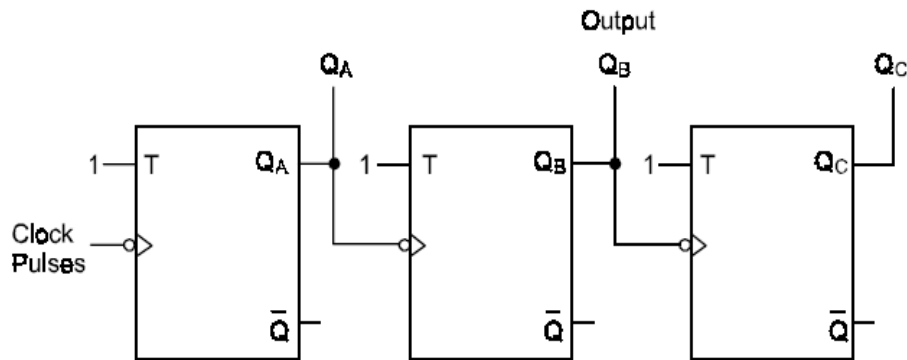


Fig. 3-Bit binary up-counter

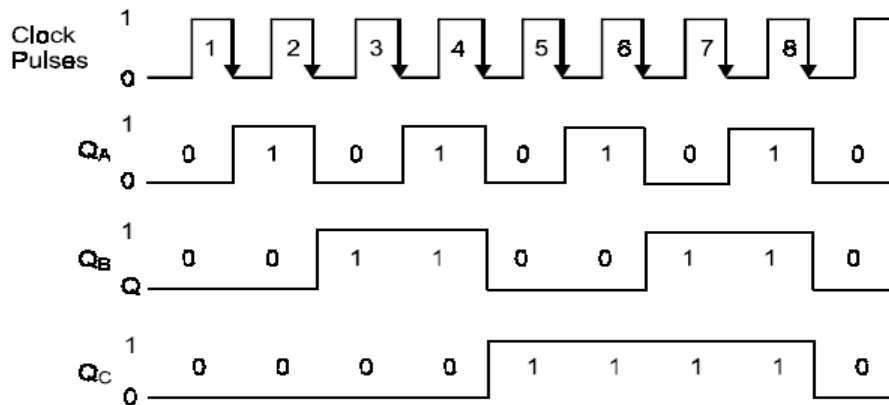


Fig. Waveform for 3-bit binary ripple up-counter

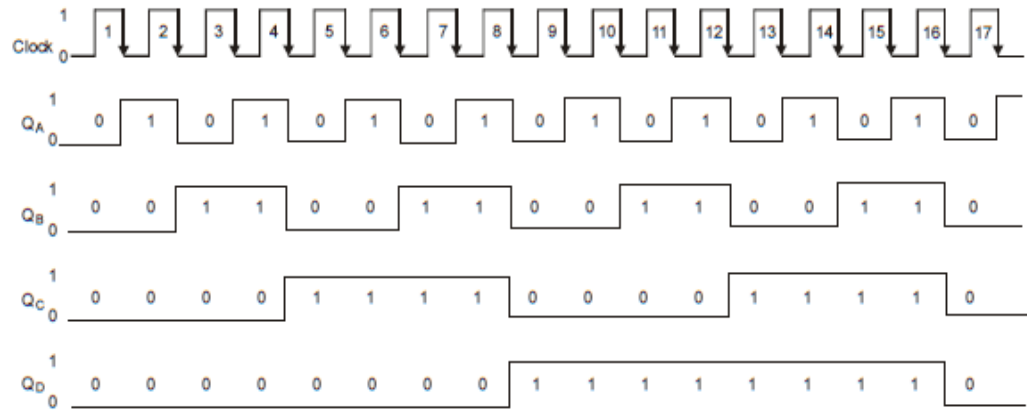
Table Count-up sequence of a 3-bit binary counter

Input pulse		Count	
	2^2	2^1	2^0
	Q_C	Q_B	Q_A
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

RECYCLE

4-Bit Binary Ripple Up-Counter

Input pulse	Count			
	2^3 Q_D	2^2 Q_C	2^1 Q_B	2^0 Q_A
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1
16 or 0	0	0	0	0



3-Bit Binary Ripple Down Counter

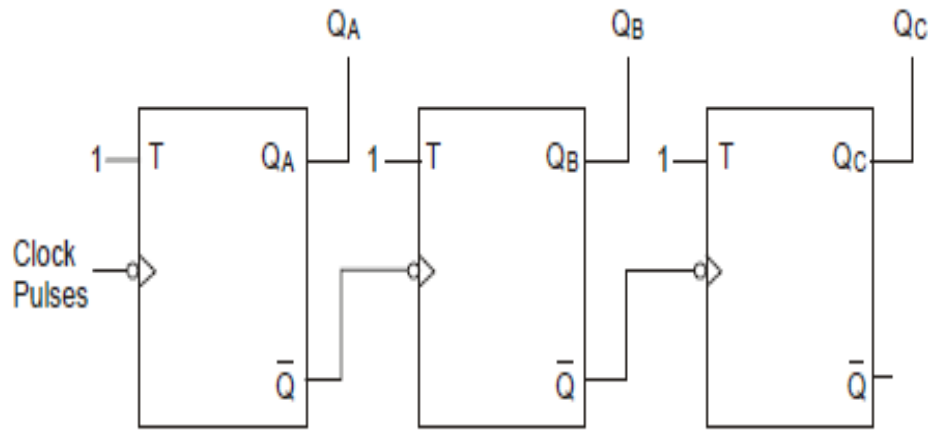


Table Count-down sequence of a 3-bit binary counter

Clock pulse	Count		
	2^2 Q_C	2^1 Q_B	2^0 Q_A
0	0	0	0
1	1	1	1
2	1	1	0
3	1	0	1
4	1	0	0
5	0	1	1
6	0	1	0
7	0	0	1
8	0	0	0

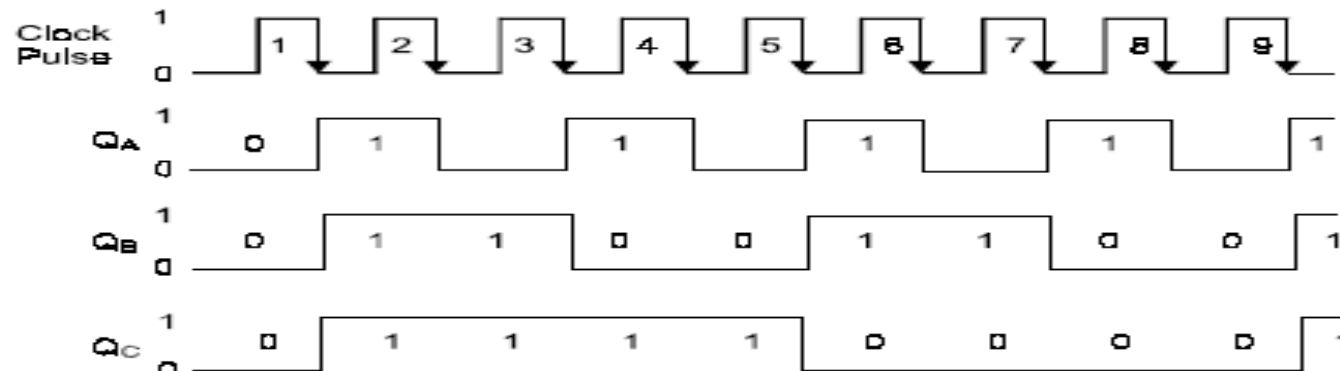
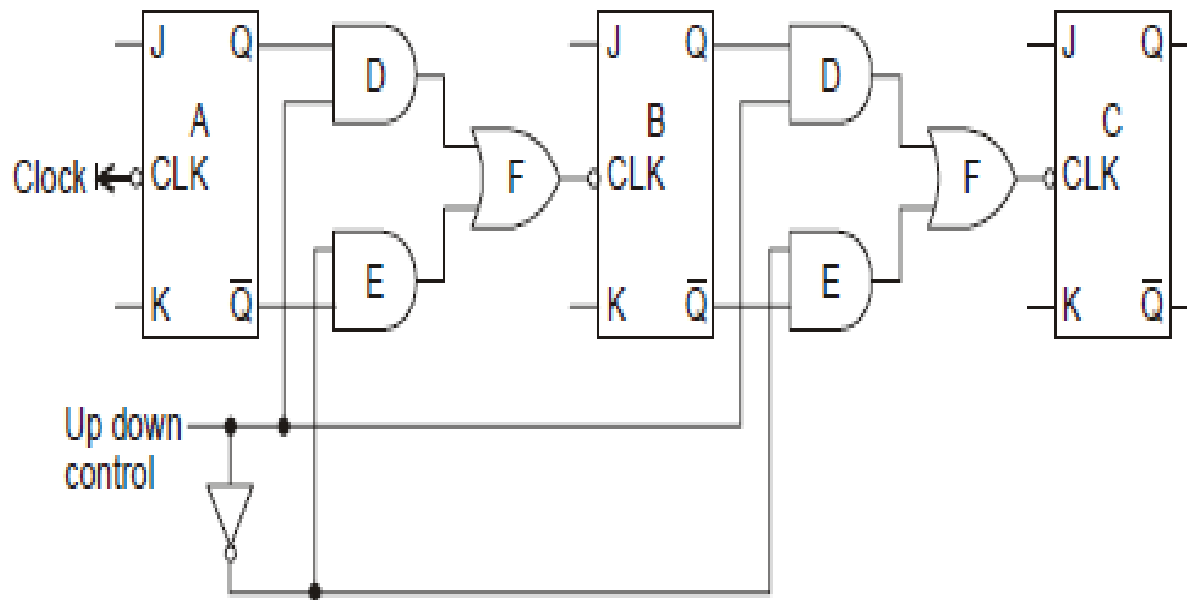
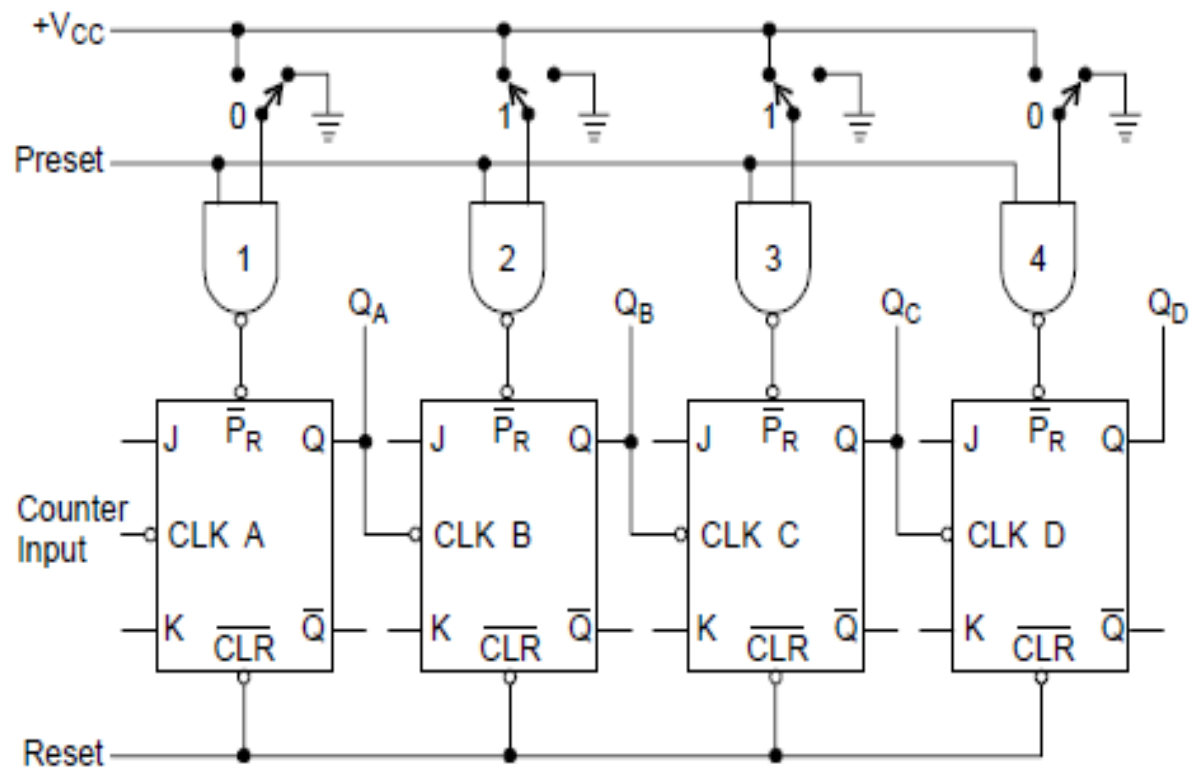


Fig. Waveform for 3-bit binary down-counter

Up-Down Counters



Reset and Preset Functions



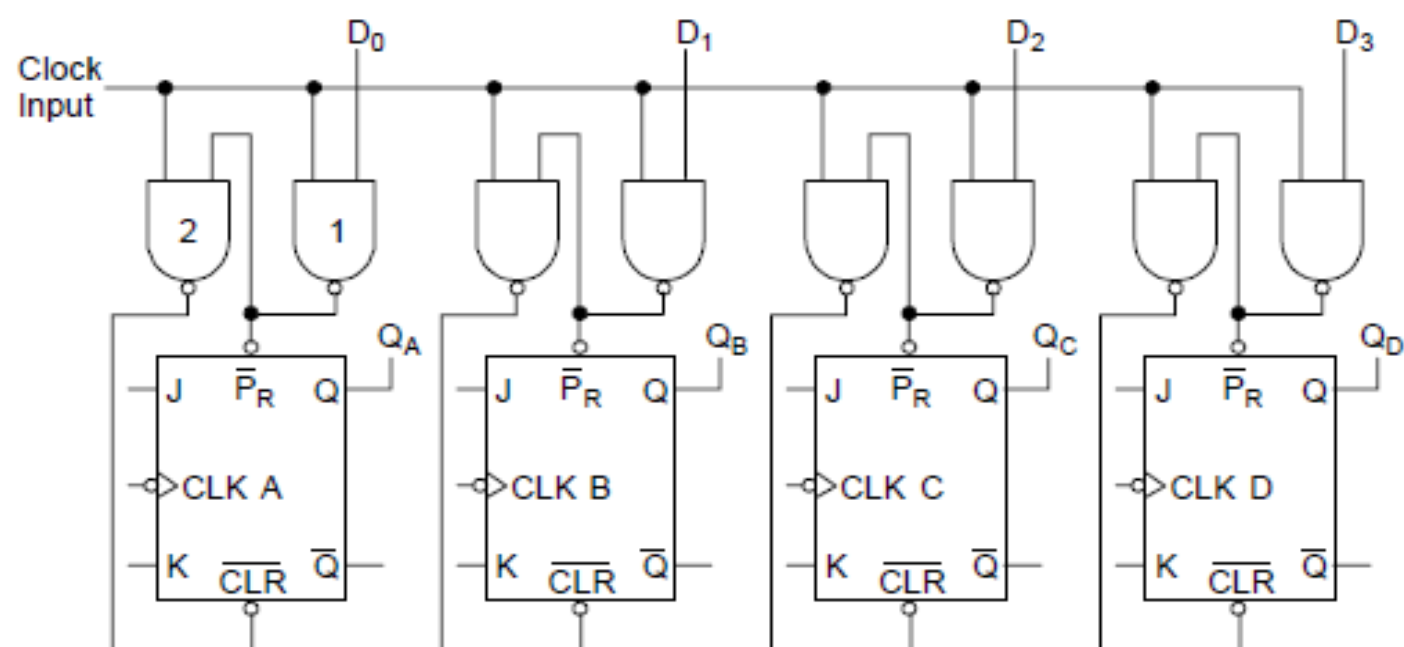


Fig. 7.20 Single pulse data transfer

Universal Synchronous Counter Stage

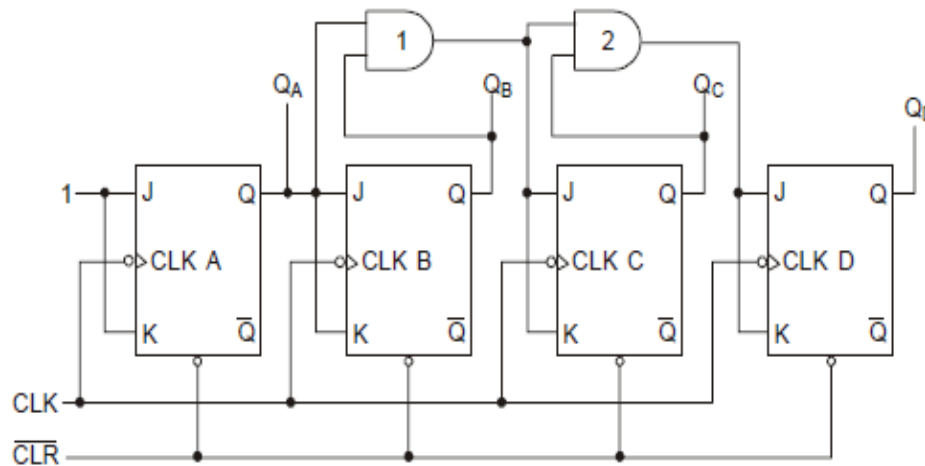


Fig. Synchronous counter

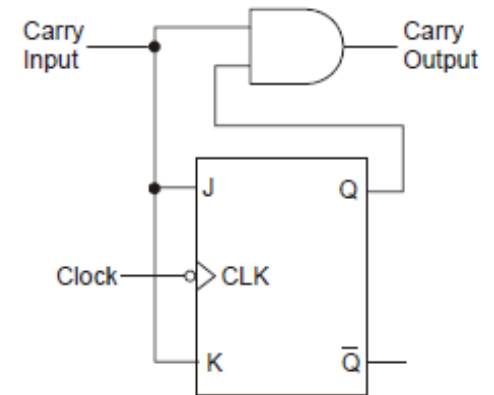


Fig. Universal counter stage block

Modulus Counters

- Many methods have been developed for designing such counters. We will consider the following:

(1) Counter Reset Method

- In this method, the counter is reset after the desired count has been reached and the count cycle starts all over again from the reset state.

(2) Logic Gating Method

- This method provides the exact count sequence required without any need to reset the counter at some stage.

(3) Counter Coupling Method

- This method is used to implement counters of the required modulus. For instance we can interconnect mod-2 and mod-3 counters to implement a modulus 3×2 or mod-6 counter.

Asynchronous Counters (Counter Reset Method)

Mod-3 Counter

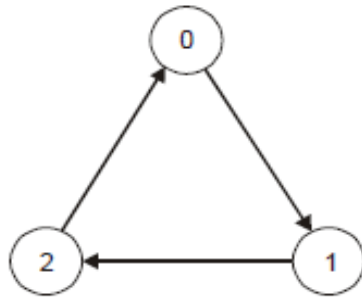


Fig. State diagram for a mod-3 counter

Table States for a two flip-flop counter

Q_A LSB	Q_B	Count value (Decimal)
0	0	0
1	0	1
0	1	2
1	1	3
0	0	0

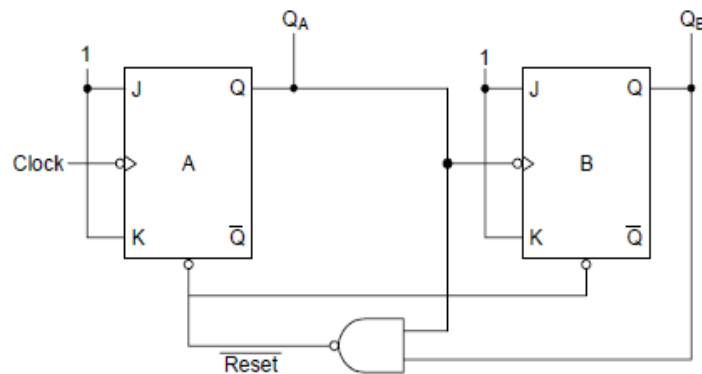


Fig. Modulo-3 counter

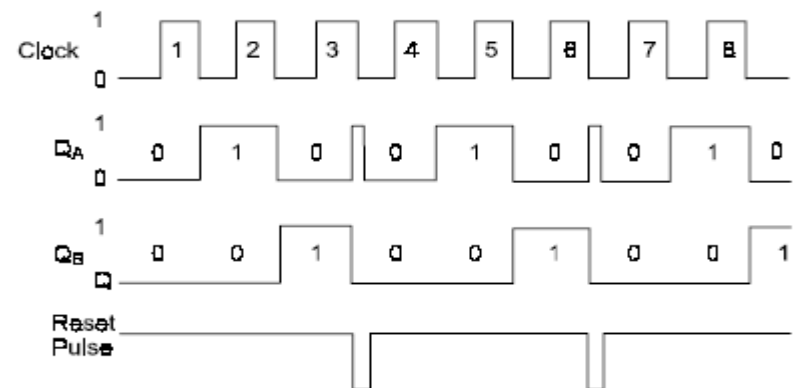


Fig. Waveform for mod-3 counter

Mod-5 Counter

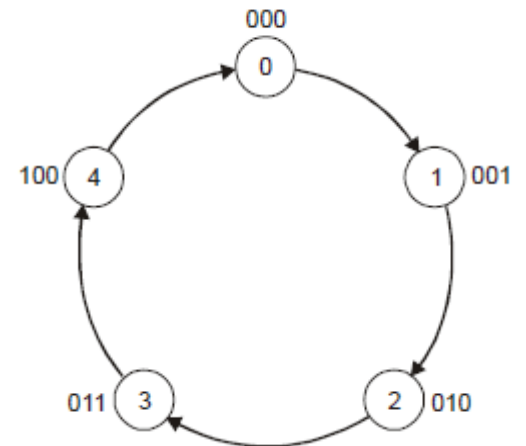


Fig. State diagram for mod-5 counter

Table Truth table for Mod-5 Counter

Q_A <i>LSB</i>	Q_B	Q_C	State
0	0	0	0
1	0	0	1
0	1	0	2
1	1	0	3
0	0	1	4
1	0	1	5
0	1	1	6 X
1	1	1	7 X

X, Don't care states

	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 11	$B\overline{C}$ 10
$\overline{A}$ 0	0 0	0 4	X 6	0 2
A 1	0 1	1 5	X 7	0 3

Modulus-5 counter

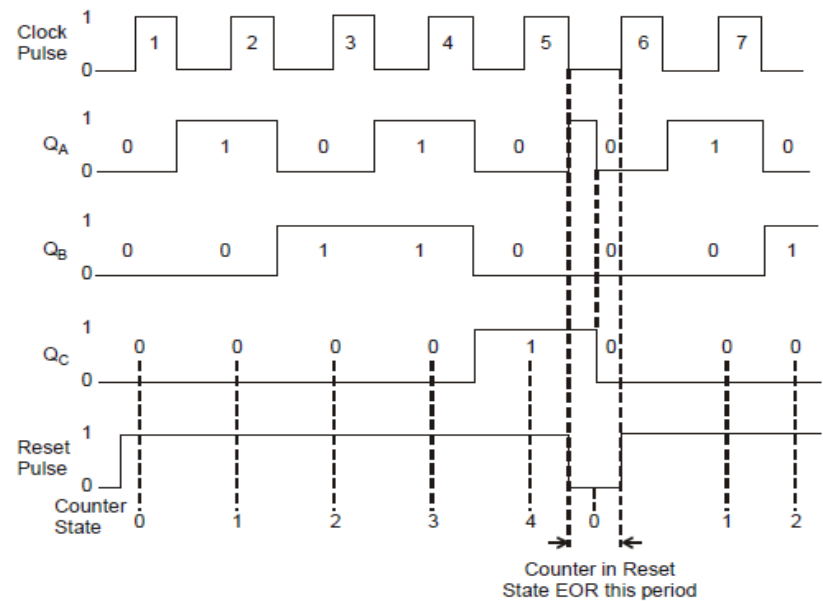
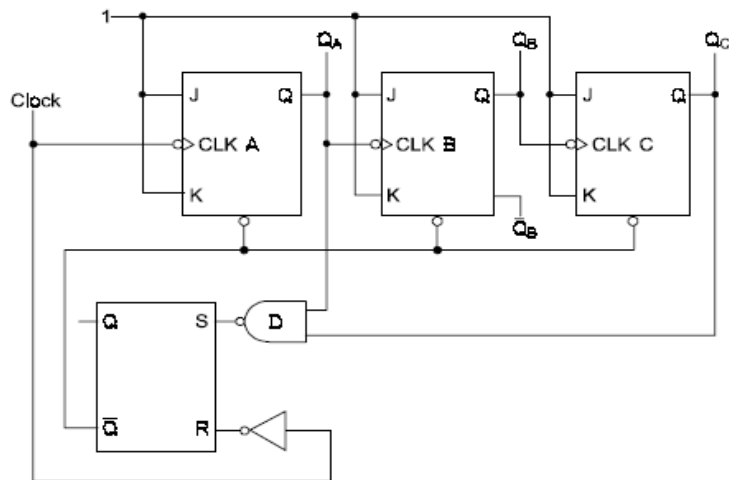


Fig. Waveform for modulus-5 asynchronous counter

Mod-10 (Decade) Counter

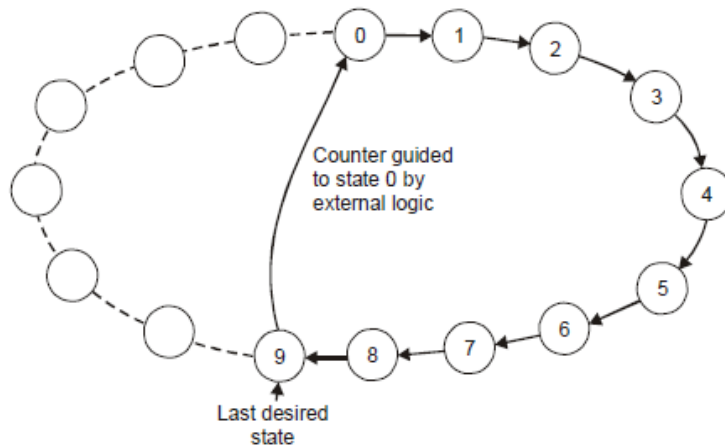


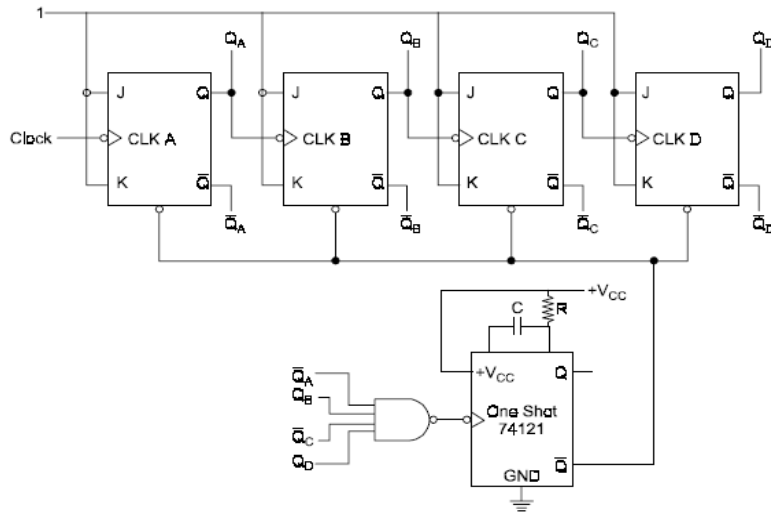
Fig. State diagram for decade counter

	$\overline{B}A_{10}$	$\overline{B}A_{11}$	BA_{11}	BA_{10}
$\overline{DC}00$	0 0	0 1	0 3	0 2
$\overline{DC}01$	0 4	0 5	0 7	0 6
$\overline{DC}11$	3 12	3 13	X 15	3 14
$\overline{DC}10$	0 8	0 9	X 11	1 10

Table Truth table for decade counter

Q_A <i>LSB</i>	Q_B	Q_C	Q_D	State
0	0	0	0	0
1	0	0	0	1
0	1	0	0	2
1	1	0	0	3
0	0	1	0	4
1	0	1	0	5
0	1	1	0	6
1	1	1	0	7
0	0	0	1	8
1	0	0	1	9
0	1	0	1	10
1	1	0	1	11 X
0	0	1	1	12 X
1	0	1	1	13 X
0	1	1	1	14 X
1	1	1	1	15 X

Decade (mod-10) asynchronous counter using count reset and pulse width control



Decade (mod-10) asynchronous counter using count reset and pulse width control

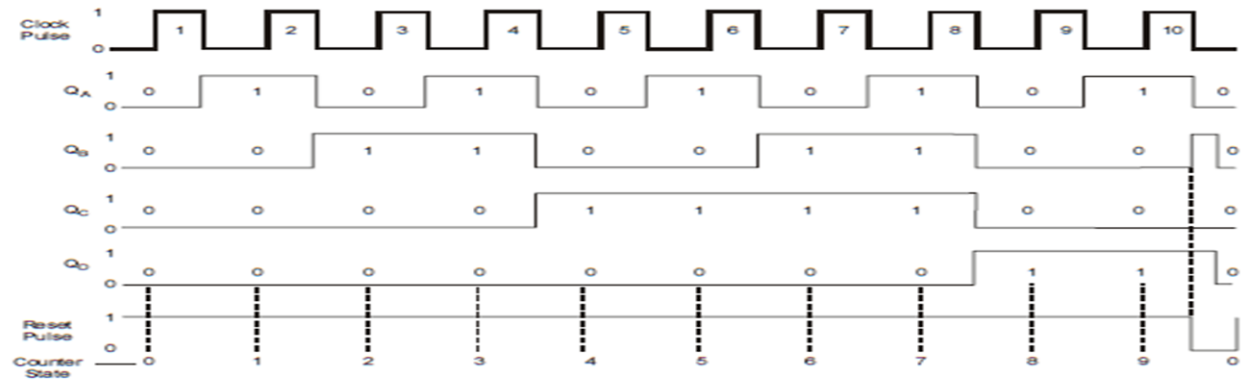


Fig. Waveform for decade counter

Logic Gating Method – Mod-3 Counter (Synchronous)

Table Truth table for Mod-3 Counter

Input pulse count	Counter states	
	A	B
0	0	0
1	1	0
2	0	1
3	0	0

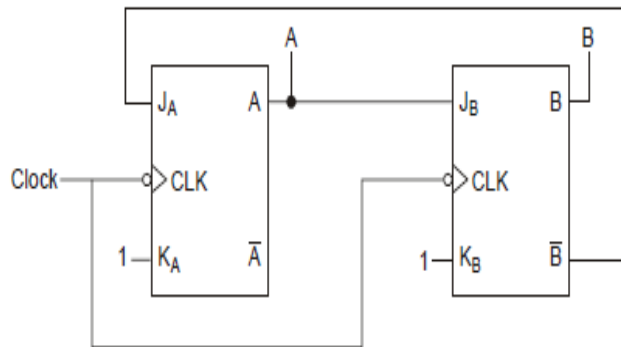


Fig. Mod-3 counter (synchronous)

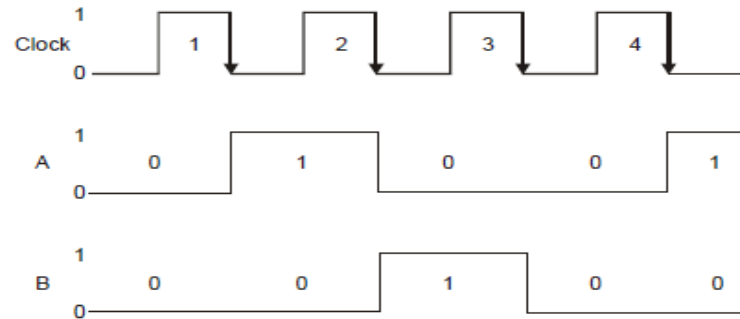


Fig. 7.33 Waveform for mod-3 counter

Mod-5 Counter (Asynchronous)

Table Truth table for Mod-5 counter

Input pulse count	Counter states		
	A	B	C
0	0	0	0
1	1	0	0
2	0	1	0
3	1	1	0
4	0	0	1
5	0	0	0

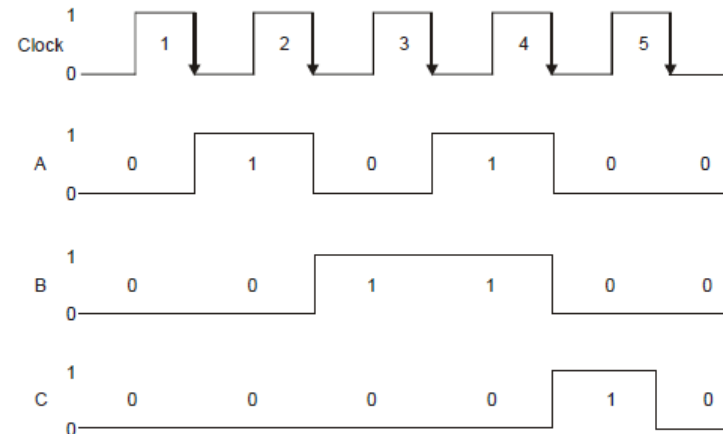


Fig. Waveform for mod-5 counter

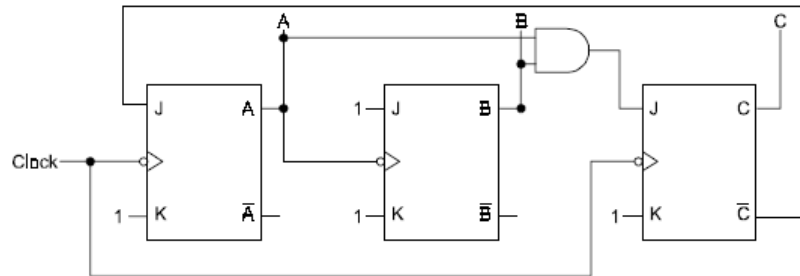


Fig. Logic diagram for mod-5 counter

Mod-10 (Decade) Counter (Asynchronous)

Table Truth Table for Decade counter

<i>Input pulse count</i>	<i>Counter states</i>			
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
0	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	1	0	0
4	0	0	1	0
5	1	0	1	0
6	0	1	1	0
7	1	1	1	0
8	0	0	0	1
9	1	0	0	1
10 (0)	0	0	0	0

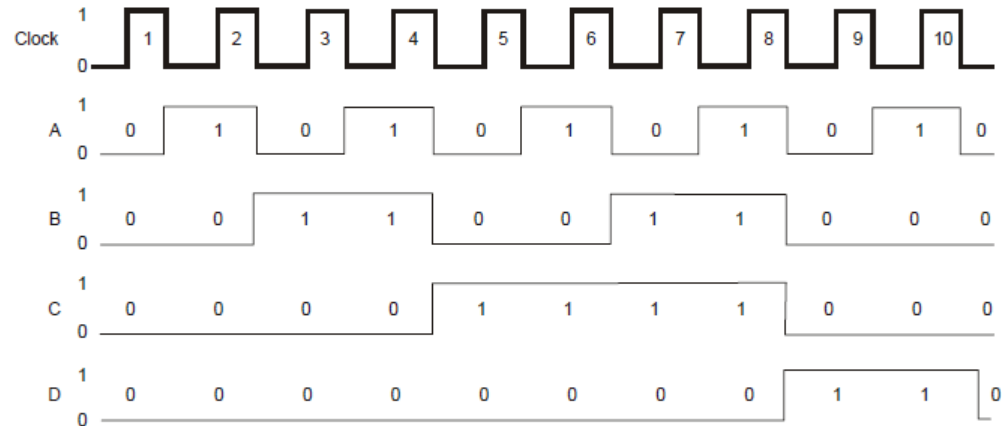


Fig. Waveform for decade counter

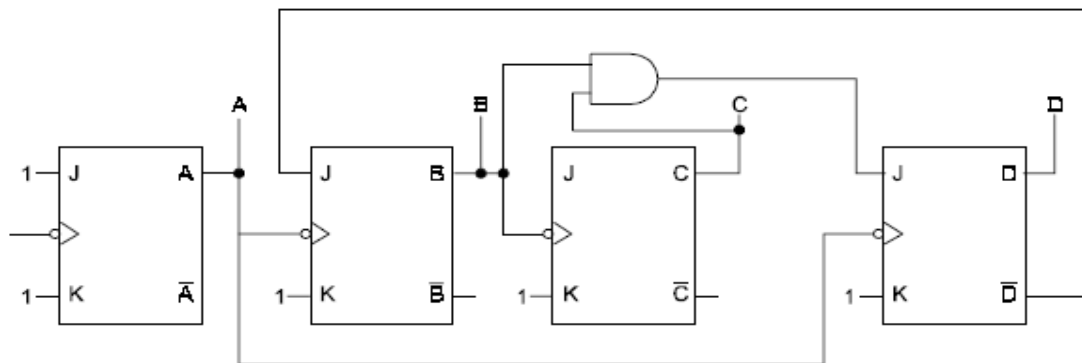


Fig. Logic diagram for decade counter

Design of Synchronous Counters

- A typical design procedure can be summed up in the following steps.
 - a) Write the desired truth table for the counter.
 - b) Write the counter transition table which should list the starting state and the subsequent states the counter is required to take up.
 - c) With the help of the excitation table and using the counter transition table, write down the input values for the J and K inputs to enable each flip-flop to attain the output state as required by the transition table.
 - d) Prepare Karnaugh maps for the J and K inputs of each stage.
 - e) Derive Boolean algebra expressions for each of the inputs to the flip-flops.
 - f) Draw the synchronous counter circuit incorporating the J and K input values obtained from the above steps.

We will take up a specific case to illustrate the above procedure.

Mod-3 Synchronous Counter

Table Excitation table for JK flip-flop

Present state	Next state	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Table Counter design table

Counter state		Flip-flop inputs			
A	B	A		B	
		J_A	K_A	J_B	K_B
0	0	1	X	0	X
1	0	X	1	1	X
0	1	0	X	X	1
0	0				

$$\begin{aligned} J_A &= \bar{B} \\ K_A &= 1 \\ J_B &= A \\ K_B &= 1 \end{aligned}$$

	0	1
$\bar{A}$ 0	1	0
	0	2
A 1	X	X
	1	3

(a)
Map for J_A
 $J_A = \bar{B}$

	0	1
$\bar{A}$ 0	X	X
	0	2
A 1	1	X
	1	3

(b)
Map for K_A
 $K_A = 1$

	$\bar{B}$ 0	B 1
$\bar{A}$ 0	X	X
	0	2
A 1	1	X
	1	3

(c)
Map for J_B
 $J_B = A$

	$\bar{B}$ 0	B 1
$\bar{A}$ 0	X	1
	0	2
A 1	X	X
	1	3

(d)
Map for K_B
 $K_B = 1$

Mod-5 Counter (Synchronous)

Table Truth table for Mod-5 counter

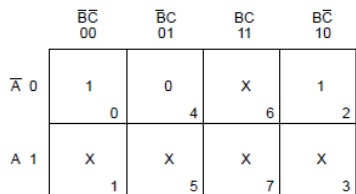
<i>Input pulse count</i>	<i>Counter states</i>		
	<i>A</i>	<i>B</i>	<i>C</i>
0	0	0	0
1	1	0	0
2	0	1	0
3	1	1	0
4	0	0	1
5	0	0	0

Table **Counter design table**

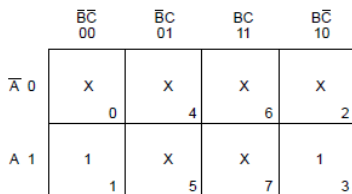
Counter state		Flip-flop inputs			
A	B	A		B	
		J_A	K_A	J_B	K_B
0	0	1	X	0	X
1	0	X	1	1	X
0	1	0	X	X	1
0	0				

Table **Counter design table for Mod-5 counter**

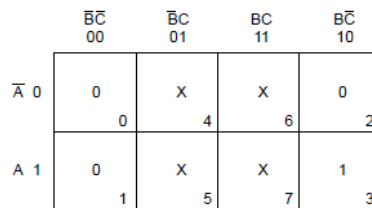
Input pulse count	Counter states			Flip-flop inputs					
	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	1	X	0	X	0	X
1	1	0	0	X	1	1	X	0	X
2	0	1	0	1	X	X	0	0	X
3	1	1	0	X	1	X	1	1	X
4	0	0	1	0	X	0	X	X	1
5 (0)	0	0	0						



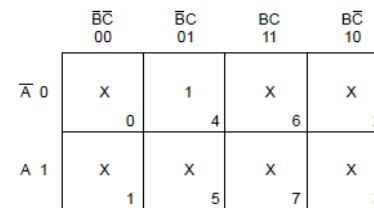
(a) Map for $\overline{J_A}$
 $J_A = \overline{C}$



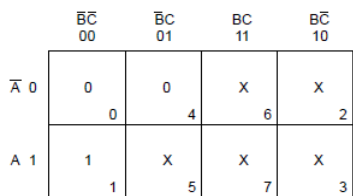
(b) Map for K_A
 $K_A = 1$



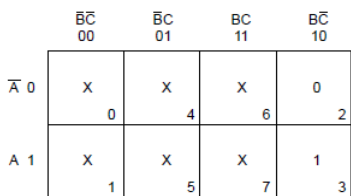
(e) Map for J_C
 $J_C = AB$



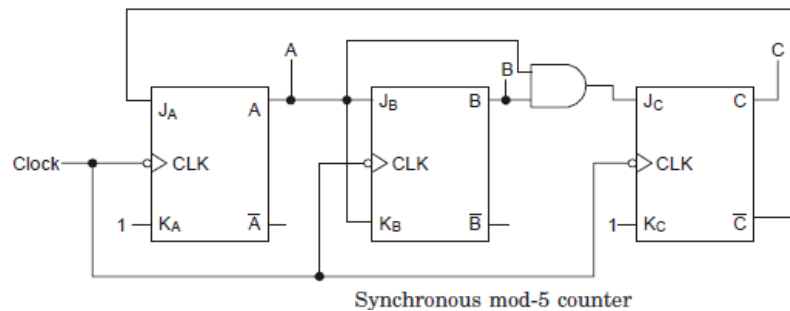
(f) Map for K_C
 $K_C = 1$



(c) Map for J_B
 $J_B = A$



(d) Map for K_B
 $K_B = A$



Synchronous mod-5 counter

Mod-6 Counter (Synchronous)

Table Counter design table for Mod-6 counter

Input pulse count	Counter states			Flip-flop inputs					
	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	1	X	0	X	0	X
1	1	0	0	X	1	1	X	0	X
2	0	1	0	1	X	X	0	0	X
3	1	1	0	X	1	X	1	1	X
4	0	0	1	1	X	0	X	X	0
5	1	0	1	X	1	0	X	X	1
6 (0)	0	0	0						

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	1	1	X	1
	0	4	6	2
A 1	X	X	X	X
	1	5	7	3

(a) Map for J_A
 $J_A = 1$

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	X	X	X	X
	0	4	6	2
A 1	1	1	X	1
	1	5	7	3

(b) Map for K_A
 $K_A = 1$

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	0	0	X	X
	0	4	6	2
A 1	1	0	X	X
	1	5	7	3

(c) Map for J_B
 $J_B = AC$

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	X	X	X	0
	0	4	6	2
A 1	X	X	X	1
	1	5	7	3

(d) Map for K_B
 $K_B = A$

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	0	X	X	0
	0	4	6	2
A 1	0	X	X	1
	1	5	7	3

(e) Map for J_C
 $J_C = AB$

$\overline{A}$ 0	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	$B\overline{C}$ 10	BC 11
	X	0	X	X
	0	4	6	2
A 1	X	1	X	X
	1	5	7	3

(f) Map for K_C
 $K_C = A$

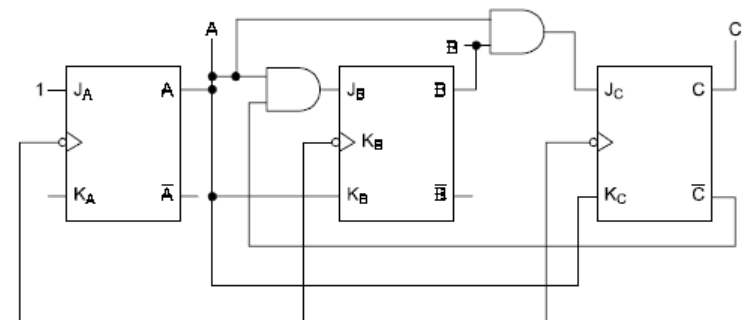
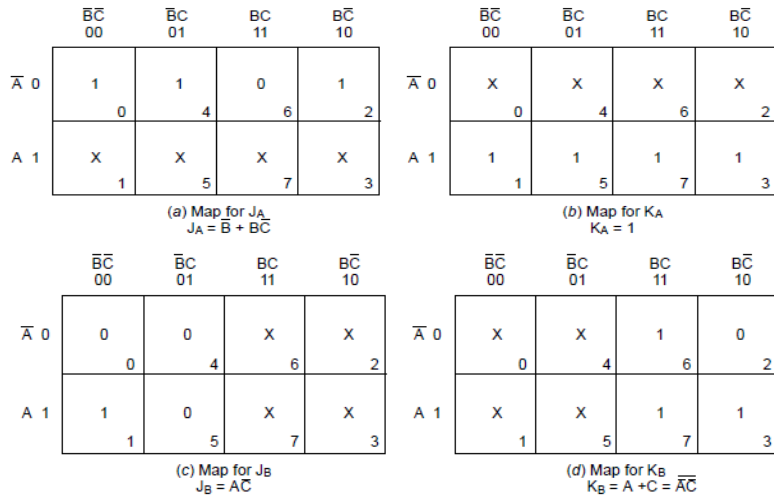


Fig. Synchronous mod-6 counter

Counter states			Flip-flop inputs					
A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	1	1	0	X	X	1	X	1
1	1	1	X	1	X	1	X	1
0	0	0						

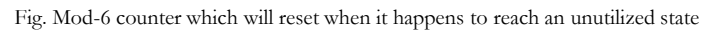


Counter states			Flip-flop inputs					
A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	1	1	0	X	X	1	X	1
1	1	1	X	1	X	1	X	1
0	0	0						

	$\overline{B}\overline{C}$ 00	$\overline{B}C$ 01	BC 11	$B\overline{C}$ 10
$\overline{A}$ 0	X 0	0 4	1 6	X 2
A 1	X 1	1 5	1 7	X 3

(e) Map for J_C
 $J_C = AB$

(f) Map for K_C
 $K_C = A + B = \overline{\overline{A}\overline{B}}$



Ring Counter

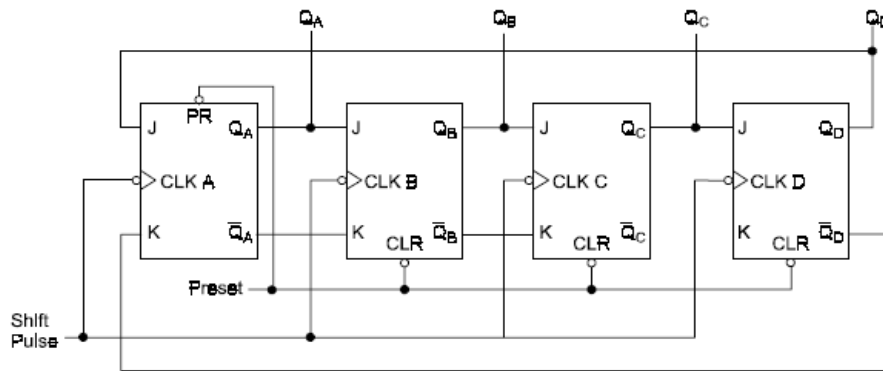


Fig. 7.48 Ring counter

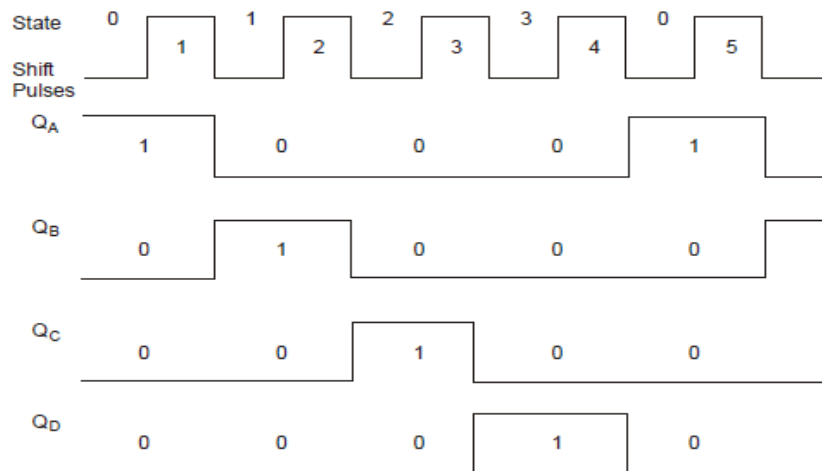


Table 7.15 Ring counter states

States	Counter output			
	Q_A	Q_B	Q_C	Q_D
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1
5	1	0	0	0

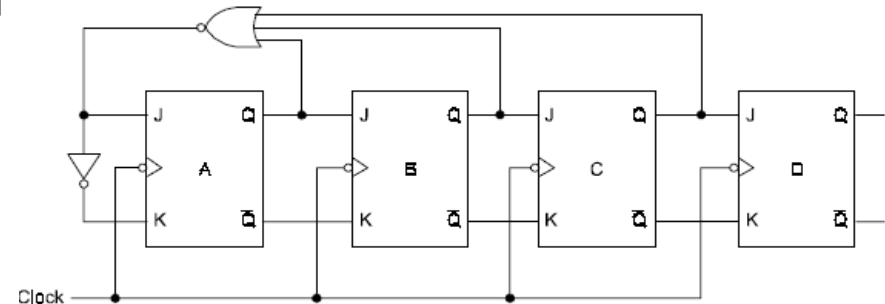
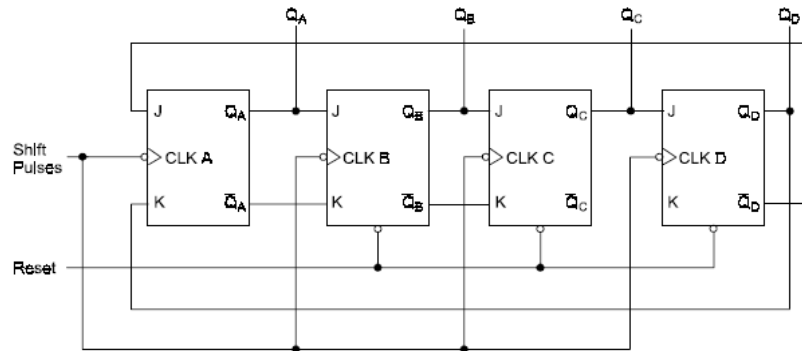


Fig. 7.50 Ring counter with correcting circuit

Johnson Counter



State	Q_D	Q_C	Q_B	Q_A	Binary equivalent	Output decoding
1	0	0	0	0	0	$\bar{A}\bar{D}$
2	0	0	0	1	1	$A\bar{B}$
3	0	0	1	1	3	$B\bar{C}$
4	0	1	1	1	7	$C\bar{D}$
5	1	1	1	1	15	AD
6	1	1	1	0	14	$\bar{A}B$
7	1	1	0	0	12	$\bar{B}C$
8	1	0	0	0	8	$\bar{C}D$

Table 7.17 Invalid states

Q_D	Q_C	Q_B	Q_A	Binary equivalent
0	1	0	0	4
1	0	0	1	9
0	0	1	0	2
0	1	0	1	5
1	0	1	1	11
0	1	1	0	6
1	1	0	1	13
1	0	1	0	10

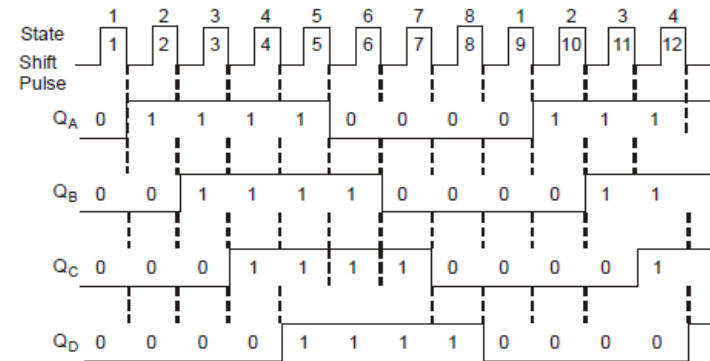


Fig. 7.52 Waveforms for a 4-stage Johnson counter

Self-starting and self-correcting Johnson counter

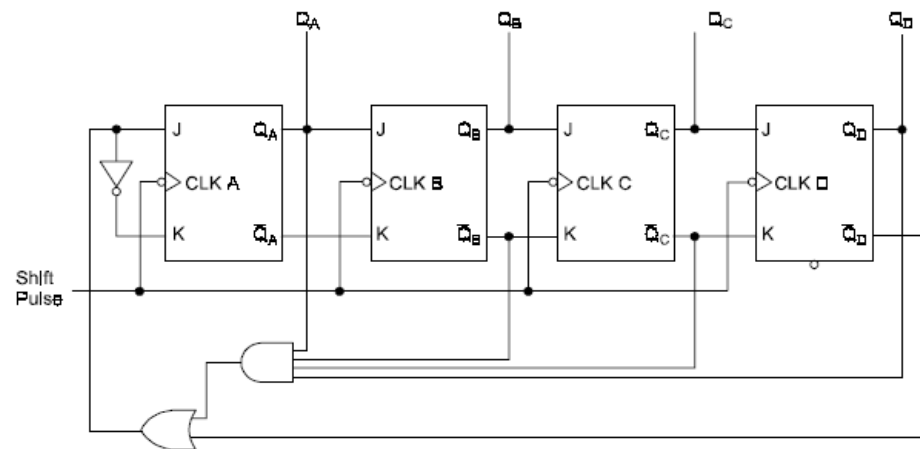


Fig. 7.53 Self-starting and self-correcting Johnson counter

Five-stage Johnson Counter

State	E	D	C	B	A	Output decoding
1	0	0	0	0	0	$\overline{A}\overline{E}$
2	0	0	0	0	1	$A\overline{B}$
3	0	0	0	1	1	$B\overline{C}$
4	0	0	1	1	1	$C\overline{D}$
5	0	1	1	1	1	$D\overline{E}$
6	1	1	1	1	1	$A E$
7	1	1	1	1	0	$\overline{A}B$
8	1	1	1	0	0	$\overline{B}C$
9	1	1	0	0	0	$\overline{C}D$
10	1	0	0	0	0	$\overline{D}E$

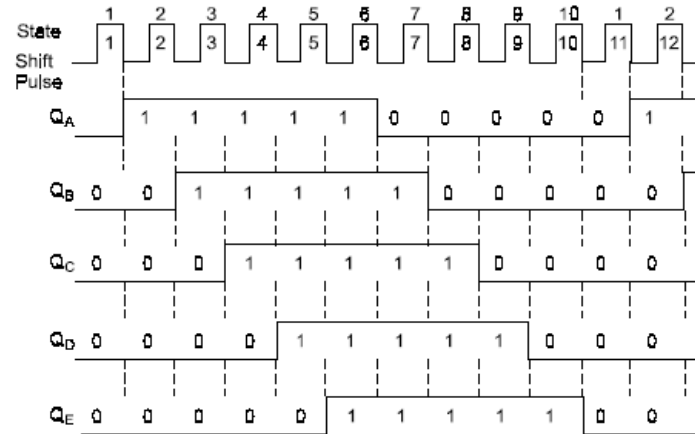
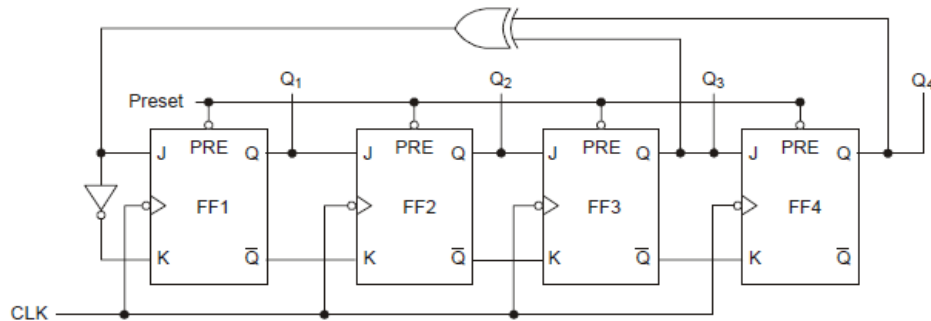


Fig. 7.54 Waveform for a 5-stage Johnson counter

Ring Counter Applications

- Ring counters find many applications as
 - (1) Frequency dividers
 - (2) Counters
 - (3) Code generators and
 - (4) Period and sequence generators

Feedback Counters



Four-stage feedback counter

Input		Output
A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

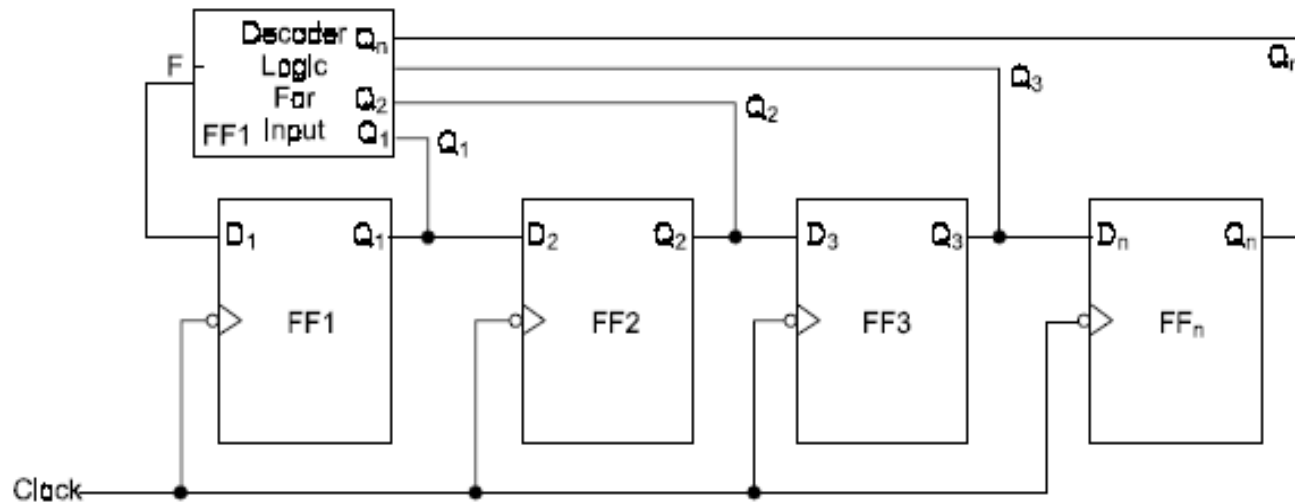
Table 7.20 Count sequence for 4-stage feedback counter

Clock input	Output			
	Q ₁	Q ₂	Q ₃	Q ₄
0	1	1	1	1
1	0	1	1	1
2	0	0	1	1
3	0	0	0	1
4	1	0	0	0
5	0	1	0	0
6	0	0	1	0
7	1	0	0	1
8	1	1	0	0
9	0	1	1	0
10	1	0	1	1
11	0	1	0	1
12	1	0	1	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

Table 7.19 Feedback terms for counter design

No. of stage	Feedback stage			
2		Q ₁	Q ₂	
3		Q ₂	Q ₃	
4		Q ₃	Q ₄	
5		Q ₃	Q ₅	
6		Q ₅	Q ₆	
7		Q ₆	Q ₇	
8	Q ₄	Q ₅	Q ₆	Q ₈
9		Q ₅	Q ₉	
10		Q ₇	Q ₁₀	
11		Q ₉	Q ₁₁	
12	Q ₆	Q ₈	Q ₁₁	Q ₁₂

Basic structure of a sequence generator



- Thank you!